



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

# DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 41 : Prompting - I



**PROF. PAWAN GOYAL**

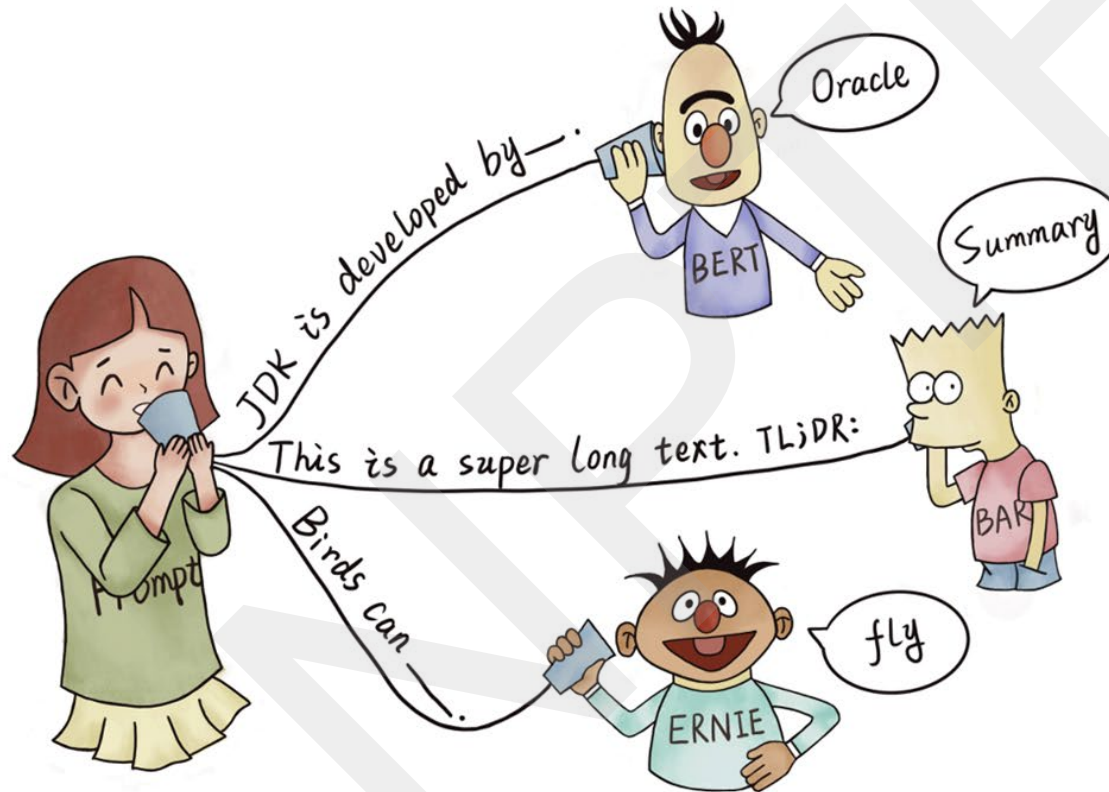
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

## CONCEPTS COVERED

- What is Prompting?
- Little bit on Prompt Engineering
- Zero-shot vs Few-shot Prompting

# What is Prompting?

Encouraging a pre-trained model to make particular predictions by providing a textual “prompt” specifying the task to be done.



# Basic Prompting



- Append a textual string to the beginning of the sequence and complete

x = **When a dog sees a squirrel, it will usually**

(GPT-2 Small)

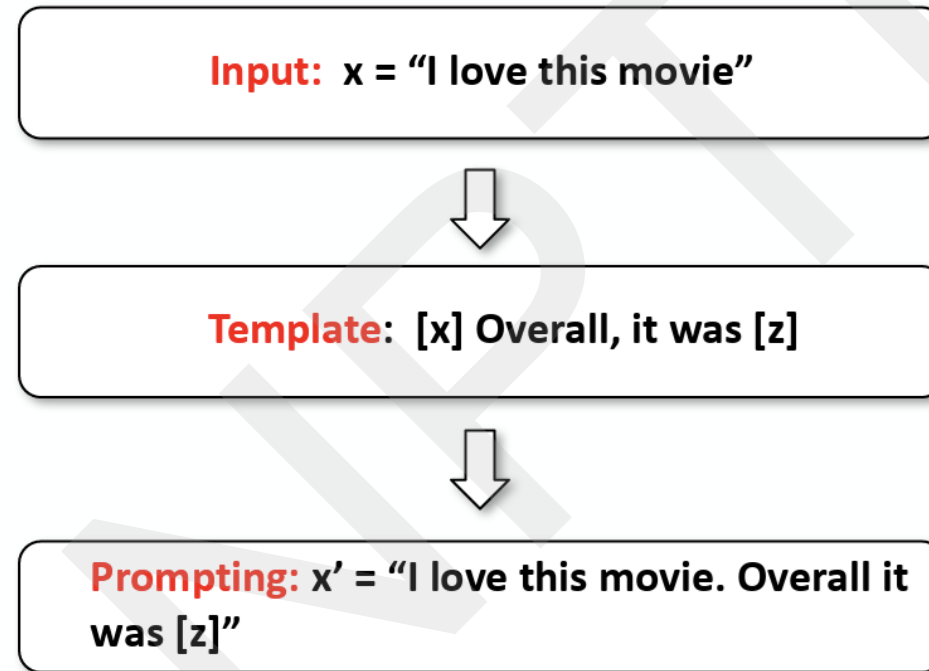
be afraid of anything unusual. As an exception, that's when a squirrel is usually afraid to bite.

(GPT-2 XL)

lick the squirrel. It will also touch its nose to the squirrel on the tail and nose if it can.

# Prompt Templates

A template where you fill in with an actual input



# Prompt Templates: Examples

| Type            | Task                | Input ([X])                                        | Template                       | Answer ([Z])                         |
|-----------------|---------------------|----------------------------------------------------|--------------------------------|--------------------------------------|
| Text CLS        | Sentiment           | I love this movie.                                 | [X] The movie is [Z].          | great<br>fantastic<br>...            |
|                 | Topics              | He prompted the LM.                                | [X] The text is about [Z].     | sports<br>science<br>...             |
|                 | Intention           | What is taxi fare to Denver?                       | [X] The question is about [Z]. | quantity<br>city<br>...              |
| Text-span CLS   | Aspect<br>Sentiment | Poor service but good food.                        | [X] What about service? [Z].   | Bad<br>Terrible<br>...               |
| Text-pair CLS   | NLI                 | [X1]: An old man with ...<br>[X2]: A man walks ... | [X1]? [Z], [X2]                | Yes<br>No<br>...                     |
| Tagging         | NER                 | [X1]: Mike went to Paris.<br>[X2]: Paris           | [X1] [X2] is a [Z] entity.     | organization<br>location<br>...      |
| Text Generation | Summarization       | Las Vegas police ...                               | [X] TL;DR: [Z]                 | The victim ...<br>A woman ...<br>... |
|                 | Translation         | Je vous aime.                                      | French: [X] English: [Z]       | I love you.<br>I fancy you.<br>...   |

Examples of input, template, and answer for different tasks. In the Type column, “CLS” is an abbreviation for “classification”

# Answer Prediction

Given a prompt, predict the answer

**Prompting:**  $x' =$  "I love this movie. Overall it was [z]"



**Predicting:**  $x' =$  "I love this movie. Overall it was fantastic"

Based on the answer, select the actual output

**For instance:**

- Taking the output as-is
- Formatting the output for easy visualization
- Selecting only parts of the output that you want to use
- Mapping the outputs to other actions

# Output Selection

From a longer response, select the information indicative of an answer

**Predicting:**  $x' =$  "I love this movie. Overall it was a **movie that was simply fantastic**"



**Extraction:** **fantastic**

*Various methods for extraction*

- Classification: identify keywords
- Regression/numerical problems: identify numbers
- Code: pull out code snippets in triple-backticks

# Output Mapping

- Given an answer, map it into a class label or continuous value

Extraction: fantastic

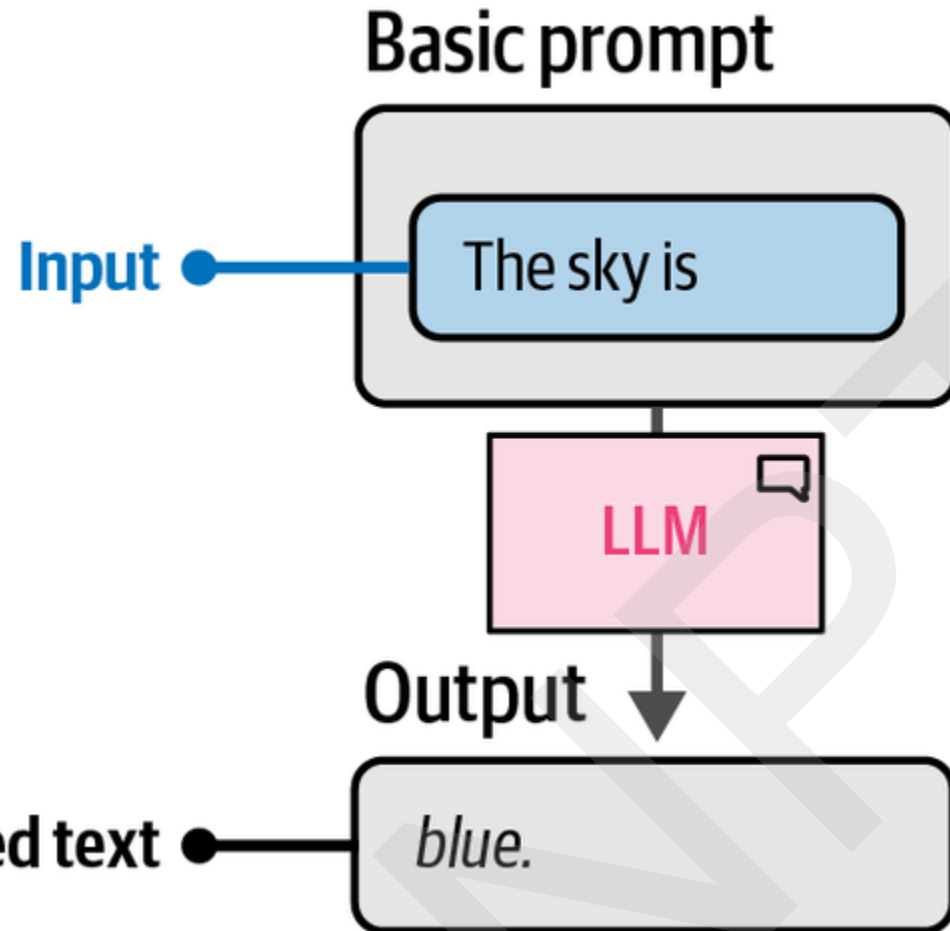


Mapping: fantastic => Positive

- Often map many extracted words onto a single class

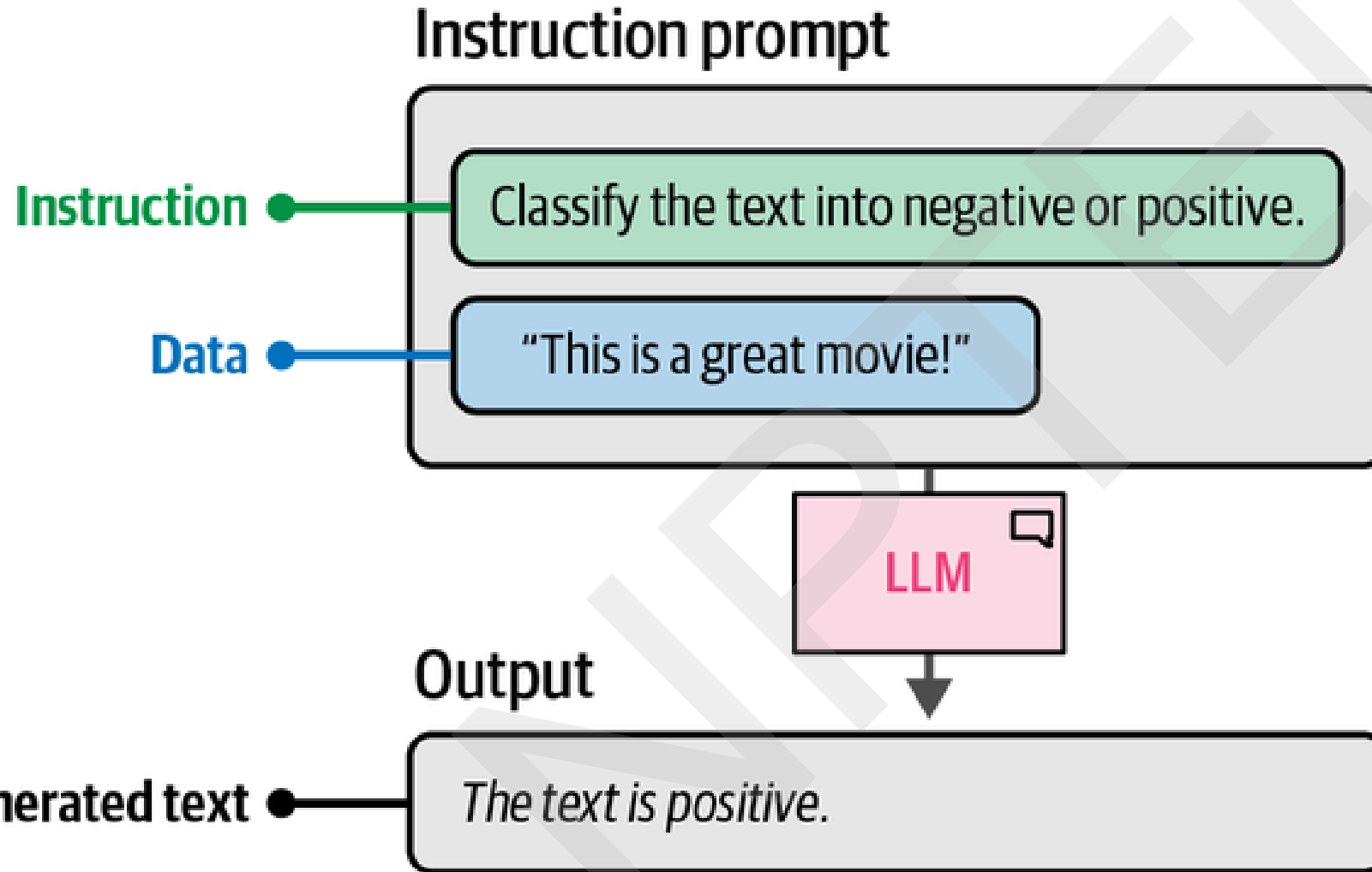


# Prompt Variations: Basic Prompt



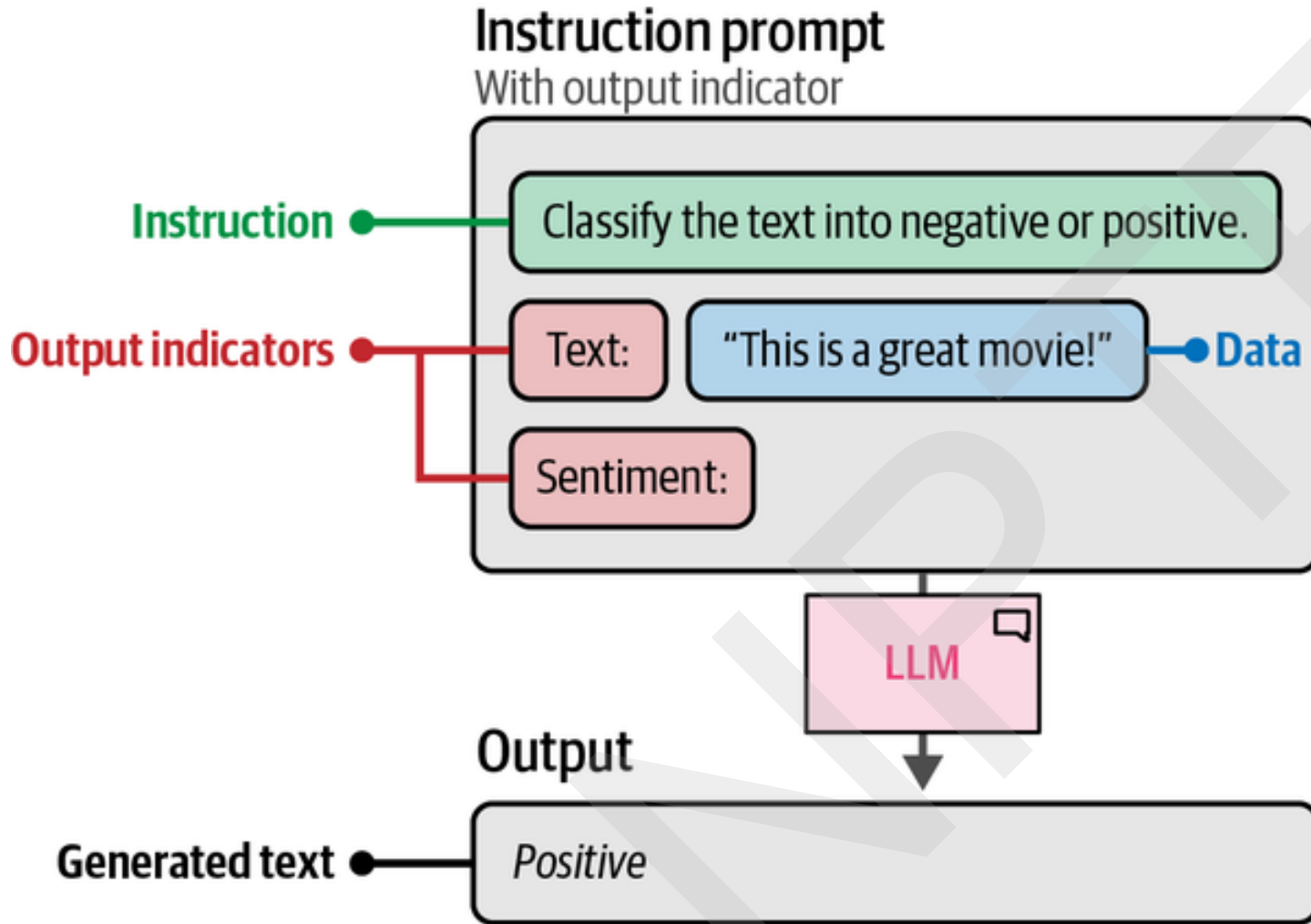
Basic example of a prompt. No instruction is given. So, the LLM will simply try to complete the sentence.

# Prompt Variations: Instruction Prompt



Two components of a basic instruction prompt: The instruction itself and the data it refers to

# Prompt Variations: Instruction Prompt



Extending the instruction prompt with an output indicator that allows for a specific output.

# Chat Prompts Behind the Scenes

- Behind the scenes, messages are converted to token strings

## LLaMa

```
Sys. [INST]
      <<SYS>>
      You are an assistant that ...
      <</SYS>>
      [/INST]

User [INST]This movie is great.[/INST]

Asst. Positive.
```

## Alpaca

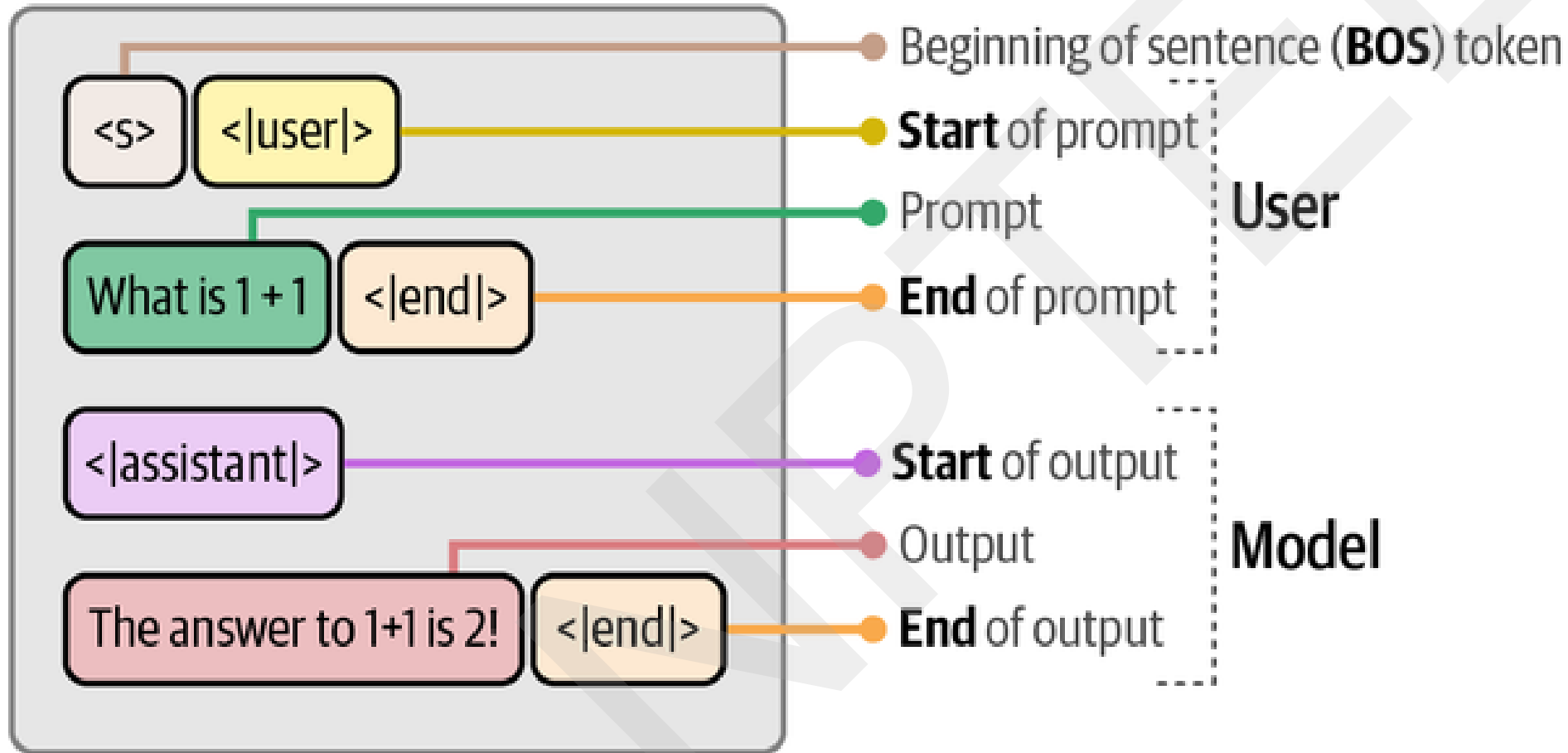
```
Sys. ### Instruction:
      You are an assistant that ...

User ### Instruction:
      This movie is great.

Asst. ### Response:
      Positive.
```

# Chat Prompts Behind the Scenes

## Phi-3 template



# Prompt Engineering Strategies

**Describing the task as clearly as possible.** *Particularly important when we want the output of the LLM to meet certain expectations.*

Suppose we are curious about climate change. A simple prompt

Tell me about climate change.

\_\_\_\_\_

The LLM may generate a response that addresses any aspect of climate change, which may not align with our specific interests. In this case, we can instead use prompts that are specific and detailed.

# Prompt Engineering Strategies

Tell me about climate change.

---

Provide a detailed explanation of the causes and effects of climate change, including the impact on global temperatures, weather patterns, and sea levels. Also, discuss possible solutions and actions being taken to mitigate these effects.

---

*specific and  
detailed prompt*

# Prompt Engineering Strategies

Provide a detailed explanation of the causes and effects of climate change, including the impact on global temperatures, weather patterns, and sea levels. Also, discuss possible solutions and actions being taken to mitigate these effects.

---

*Now suppose we intend to explain climate change to a 10-year-old child. We can adjust the above prompt further.*

Explain the causes and effects of climate change to a 10-year-old child. Talk about how it affects the weather, sea levels, and temperatures. Also, mention some things people are doing to help. Try to explain in simple terms and do not exceed 500 words.

---

# Asking LLM to follow a reasoning pattern

You are a mathematician. You will follow these detailed reasoning steps when solving math problems.

Step 1: Problem Interpretation.

The mathematician carefully listens to your query and understands the intricate details of the mathematical challenge you have presented.

Step 2: Strategy Formulation.

Drawing upon their extensive knowledge, the mathematician chooses the most effective strategy tailored to the type of math problem, whether it is algebra, calculus, or geometry.

Step 3: Detailed Calculation.

With precision and expertise, the mathematician performs the necessary calculations step by step, adhering to all mathematical principles.

Step 4: Solution Review.

Before providing the final answer, the mathematician meticulously checks the calculations for accuracy and offers a concise explanation or rationale for the solution.

You will be provided with a math problem. Please solve the problem.

{\*problem\*}

*Since solving math problems requires a detailed reasoning process, LLMs would probably make mistakes if they attempted to work out the answer directly. So we can explicitly ask LLMs to follow a given reasoning process before coming to a conclusion*

# Multiple rounds of interactions

*As a first step, we can instruct LLMs to solve the problem directly*

You will be provided with a math problem. Please solve the problem.

{\*problem\*}

\_\_\_\_\_

*As a second step, we prompt LLMs to evaluate the correctness of the answer and, if necessary, rework it to find a better solution.*

You will be provided with a math problem, along with a solution. Evaluate the correctness of this solution, and identify any errors if present. Then, work out your own solution.

Problem: {\*problem\*}

Solution: {\*solution\*}

\_\_\_\_\_

*More detailed discussion on how to improve prompting through more reasoning later*

# A complex prompt with many components

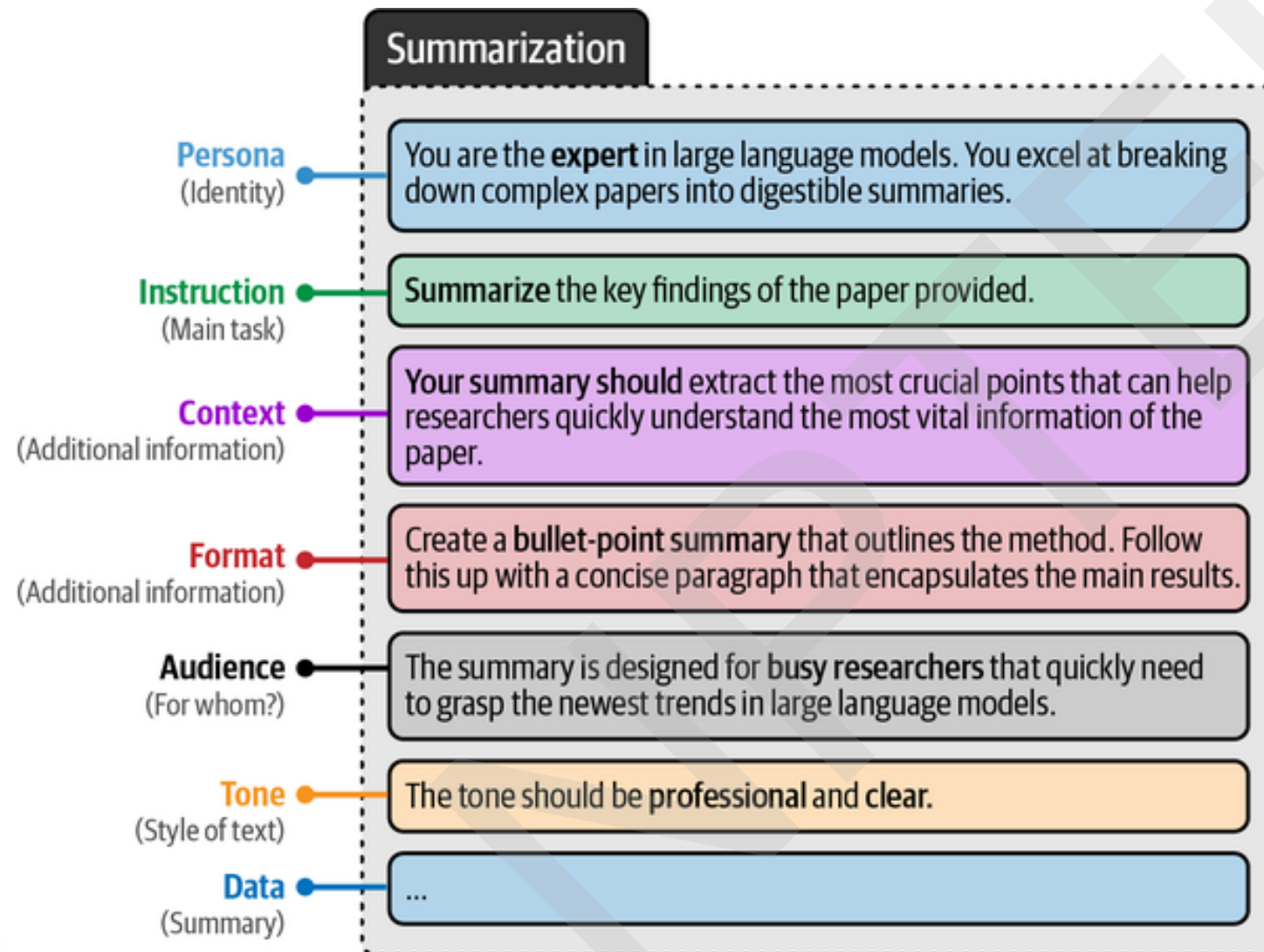


Image Source: Hands-on Large Language Models

# Zero-shot vs Few-shot Prompts

## Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: *I think the food was okay.*

Sentiment: ...

## One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: *I think the food was okay.*

Sentiment:

## Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral.**

Text: I think the food was great!

Sentiment: **Positive.**

Text: I think the food was horrible...

Sentiment: **Negative.**

Text: *I think the food was okay.*

Sentiment:

# Few-shot Prompting w/ Chat Prompts

- For OpenAI models, add "role": "system" and a "name": "example\_assistant" etc.

```
messages=[
  {
    "role": "system",
    "content": "You are an assistant that translates corporate jargon into plain English."
  },
  {
    "role": "system",
    "name": "example_user",
    "content": "New synergies will help drive top-line growth."
  },
  {
    "role": "system",
    "name": "example_assistant",
    "content": "Things working well together will increase revenue."
  },
  ...,
  {
    "role": "user",
    "content": "This late pivot means we don't have time to boil the ocean for the client deliverable."
  },
]
```

# Few-shot learning

We essentially provide a pattern that maps some inputs to the corresponding outputs. The LLM attempts to follow this pattern in making predictions, provided that the prompt includes a sufficient number of demonstrations, *although generally small*.

SYSTEM You are a helpful assistant, and are great at grammar correction.

DEMO1 You will be provided with a sentence in English. The task is to output the correct sentence.

Input: There is many reasons to celebrate.

Output: There are many reasons to celebrate.

DEMO2 You will be provided with a sentence in English. The task is to output the correct sentence.

Input: Me and my friend goes to the gym every day.

Output: My friend and I go to the gym every day.

USER You will be provided with a sentence in English. The task is to output the correct sentence.

Input: She don't like going to the park.

Output: \_\_\_\_\_

*Why does it work?*  
More on this in the next  
lecture

# REFERENCES

Foundations of Large Language Models:  
<https://arxiv.org/abs/2501.09223>

Pre-train, Prompt, and Predict: A Systematic Survey of Prompting  
Methods in Natural Language Processing

Hands-on Large Language Models



**THANK YOU**



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

# DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 42 : Prompting: Why does in-context learning work?



**PROF. PAWAN GOYAL**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

## CONCEPTS COVERED

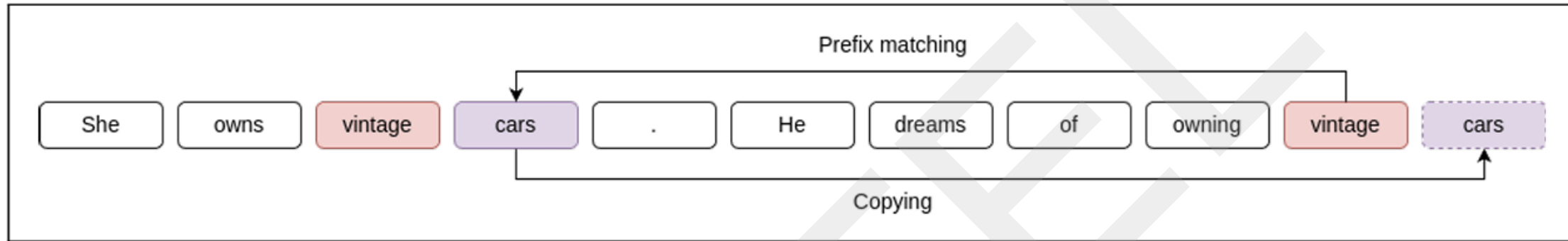
- Induction Head Hypothesis
- In-context learning and induction heads
- Some counterintuitive effects

# Why does in-context learning work?

In-context learning means language models learning to

- do new tasks
  - better predict tokens, or
  - generally reduce their loss
- 
- during the forward-pass at inference-time,
  - without any gradient-based updates to the model's parameters.

# Induction Head Hypothesis



**Figure 12.3** An induction head looking at *vintage* uses the *prefix matching* mechanism to find a prior instance of *vintage*, and the *copying* mechanism to predict that *cars* will occur again. Figure from [Crosbie and Shutova \(2022\)](#).

The function of the induction head is to predict repeated sequences.

For example if it sees the pattern  $A B \dots A$  in an input sequence, it predicts that  $B$  will follow, instantiating the pattern completion rule  $A B \dots A B$ .

# Induction Head Hypothesis



- Induction heads are the name for a circuit, which is a kind of abstract component of a network.
- The induction head circuit is part of the attention computation in transformers, discovered by looking at mini language models with only 1-2 attention heads.

# A Mathematical Framework for Transformer Circuits

## AUTHORS

Nelson Elhage<sup>\*†</sup>, Neel Nanda<sup>\*</sup>, Catherine Olsson<sup>\*</sup>, Tom Henighan<sup>†</sup>, Nicholas Joseph<sup>†</sup>, Ben Mann<sup>†</sup>, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, Chris Olah<sup>‡</sup>

<sup>\*</sup> Core Research Contributor; <sup>†</sup> Core Infrastructure Contributor; <sup>‡</sup> Correspondence to colah@anthropic.com; Author contributions statement below.

## AFFILIATION

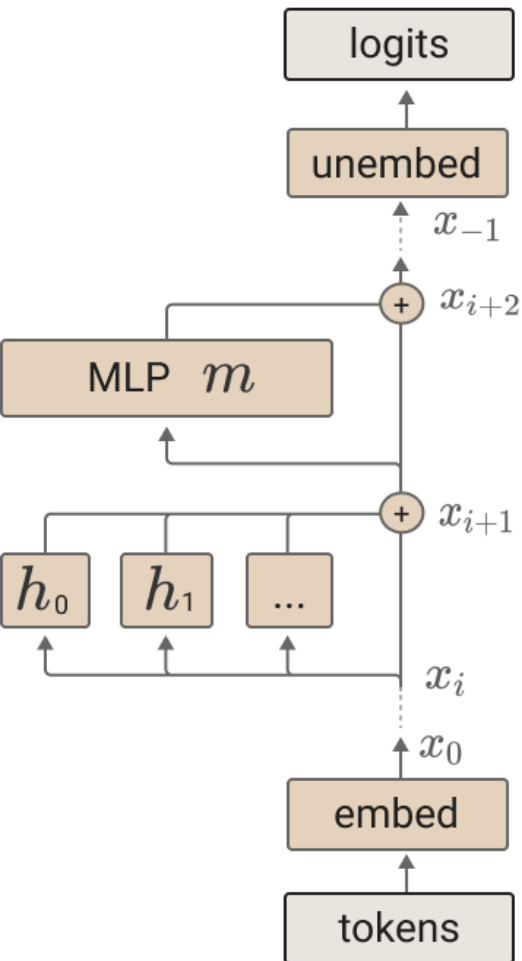
Anthropic

## PUBLISHED

Dec 22, 2021

<https://transformer-circuits.pub/2021/framework/index.html>

# Transformer Architecture



The final logits are produced by applying the unembedding.

$$T(t) = W_U x_{-1}$$

An MLP layer,  $m$ , is run and added to the residual stream.

$$x_{i+2} = x_{i+1} + m(x_{i+1})$$

Each attention head,  $h$ , is run and added to the residual stream.

$$x_{i+1} = x_i + \sum_{h \in H_i} h(x_i)$$

One  
residual  
block

Token embedding.

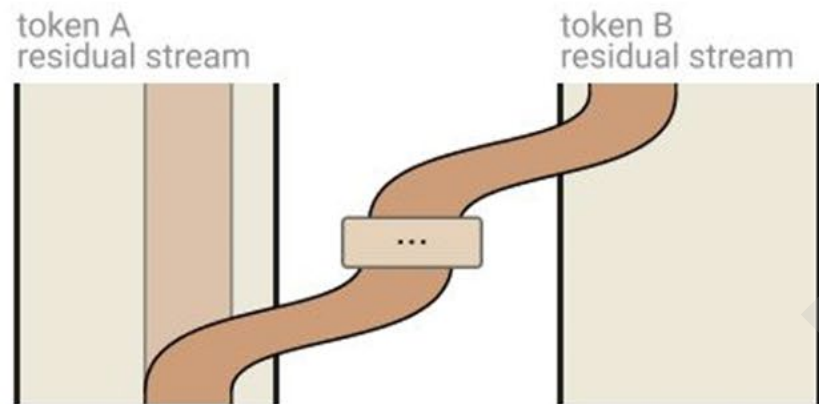
$$x_0 = W_E t$$

*Residual stream is simply the sum of the output of all the previous layers and the original embedding.*

# The residual stream

Attention heads move information across residual streams.

MLPs move information across dimensions within a residual stream.



Attention heads copy information from the residual stream of one token to the residual stream of another. They typically write to a different subspace than they read from.

# Important notions

- The central object in the transformer is the **residual stream**.
- Different heads in each layer can be thought of as **operating independently** of each other, reading and writing into the residual stream.
- Heads can compose to form **circuits**. For instance,  **$K$ -composition** is when the output of one head is used to generate the key vector in the attention calculations of a subsequent head.
- We can describe the weight matrices  $W_Q$ ,  $W_K$  and  $W_V$  as **reading from** (or **projecting from**) the residual stream, and  $W_O$  as **writing to** (or **embedding into**) the residual stream.

# Important notions

- We can think of the combined operations  $W_Q$  and  $W_K$  in terms of a single, low-rank matrix  $W_{QK} := W_Q W_K^T$ , called the **QK circuit**.
  - This matrix defines a bilinear form on the vectors in the residual stream:  
 $v_i^T W_{QK} v_j$  is the attention paid by the  $i$ th token to the  $j$ th token.
  - Conceptually, this matrix tells us **which tokens information is moved to & from** in the residual stream.
- We can think of the combined operations  $W_V$  and  $W_O$  in terms of a single matrix  $W_{OV} := W_V W_O$ , called the **OV circuit**.
  - This matrix defines a map from residual stream vectors to residual stream vectors: if  $v_j$  is the residual stream vector at the source token, then  $v_j^T W_{OV}$  is the vector that gets moved from token  $j$  to the destination token (if  $j$  is attended to).
  - Conceptually, this matrix tells us **what information is moved from a token**, if that token is attended to.

# Subspaces in the residual stream

As information moves through this model, it will be stored in 4 different subspaces.  
Each one has a different interpretation, so it will help to describe the subspaces in these terms:

|                                                                   |                              |
|-------------------------------------------------------------------|------------------------------|
| token encoding subspace (i.e. "this token is x")                  | = rows of $W_E$              |
| positional encoding subspace (i.e. "this token is at position x") | = rows of $W_{pos}$          |
| decoding subspace (i.e. "the next token will be x")               | = cols of $W_U$              |
| prev token subspace (i.e. "the previous token was x")             | = "intermediate information" |

To make the notation cleaner, I'll leave subspaces empty if they've not been used yet.

e.g. after adding token & positional encodings, we get

which I'll write as just

I am "D"  
at posn=0


$$\begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \\ \text{ } \\ \text{ } \end{bmatrix} + \begin{bmatrix} \text{ } \\ \text{at posn=0} \\ \text{ } \\ \text{ } \end{bmatrix} = \begin{bmatrix} \text{I am "D"} \\ \text{at posn=0} \\ \text{ } \\ \text{ } \end{bmatrix}$$

More notation: "A" is a one-hot encoded token vector, i.e.  $\text{"A"}^T W_E = \text{I am "A"}$

$n$  is for a one-hot encoded position vector, i.e.  $n^T W_{pos} = \text{at posn=n}$

# An example: Embedding

## Embedding



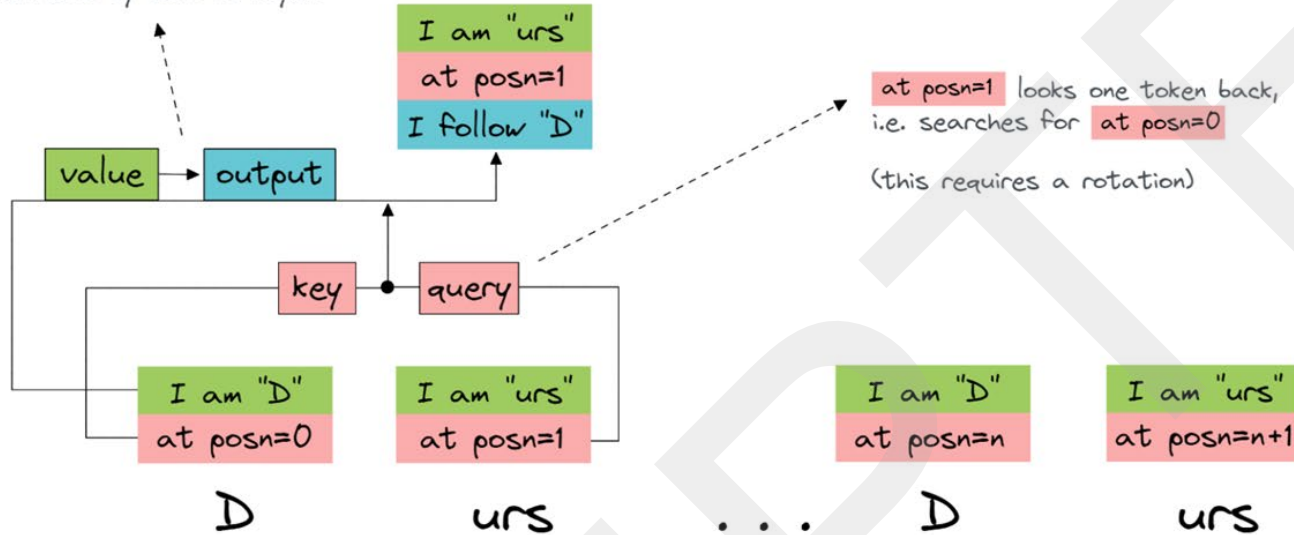
token embedding, from  $W_E$   
positional embedding, from  $W_{pos}$

D      urs      ...      D      urs      ...

# Layer 0 attention head

Summary: each token looks one position backwards, and gets the information about which token preceded it.

"I am 'D'" is converted to "I follow 'D'" and moved to the destination token, ready to be read by second layer.



## OV circuit

## QK circuit

Query vector is  $\begin{bmatrix} \text{"I am 'urs'"} \\ \text{"at posn=1"} \end{bmatrix}^T W_Q = \text{"I'm looking for posn } 1 - 1 = 0"$

Key vector is  $\begin{bmatrix} \text{"I am 'D'"} \\ \text{"at posn=0"} \end{bmatrix}^T W_K = \text{"I'm at posn } 0"$

i.e. we subtract one from the current position (this requires a rotation<sup>[3]</sup>)

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

The OV circuit reads token information, and moves it to a different subspace (so it doesn't erase the token embedding information which is already stored at the destination token).

We have:

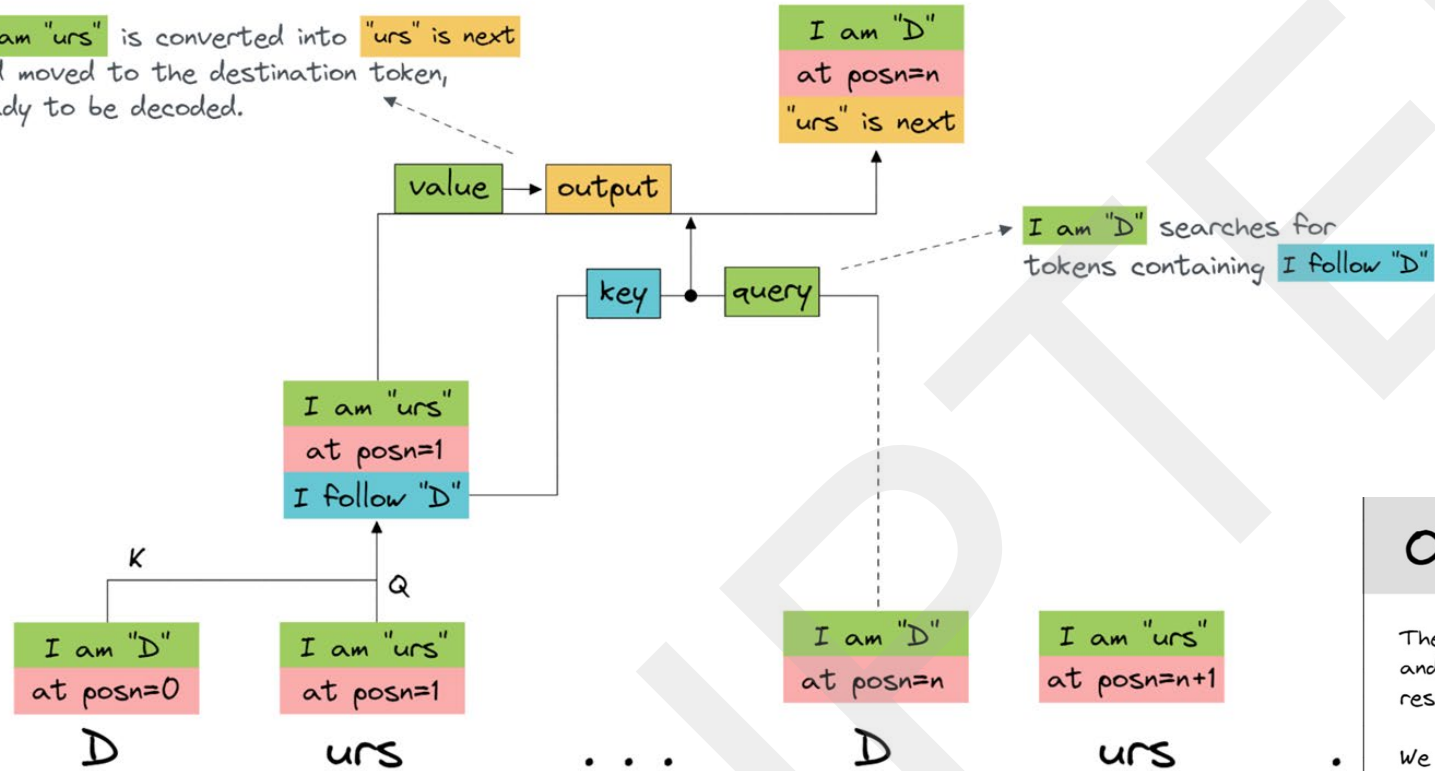
$$\begin{bmatrix} \text{"I am 'D'"} \\ \text{"at posn=0"} \end{bmatrix}^T W_V W_O = \text{"I follow 'D'"}$$

which gets added to the residual stream for the first "urs" token.

# Layer 1 attention head

Summary: the 2nd "D" token looks back for the token following the 1st "D" (which is "urs"), and uses that as its prediction.

"I am 'urs'" is converted into "urs" is next and moved to the destination token, ready to be decoded.



## QK circuit

The query is  $\begin{bmatrix} \text{I am "D"} \\ \text{at posn=n} \end{bmatrix}^T W_Q = \text{"I'm looking for a token following "D" "}$

The key is  $\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_K = \text{"I follow "D" "}$

The attention score (the dot product of the key and query vectors) is large, because the key is a good match for the query.

## OV circuit

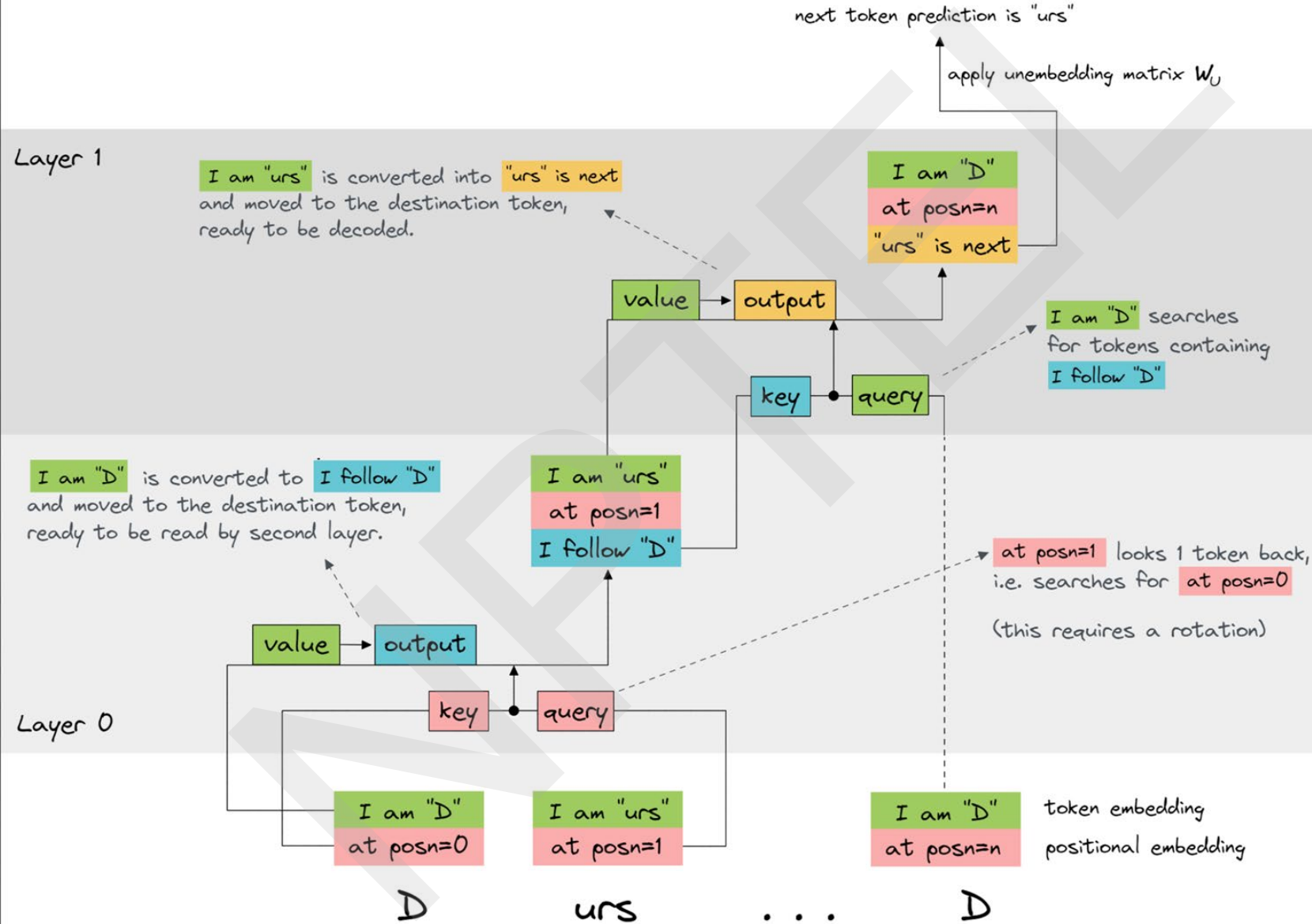
The OV circuit reads token information, and converts it into something which will result in a prediction of "urs".

We have:

$\begin{bmatrix} \text{I am "urs"} \\ \text{at posn=1} \\ \text{I follow "D"} \end{bmatrix}^T W_V W_O = \text{"urs" is next}$

which gets added to the residual stream of the second "D" token.

# Putting everything together:

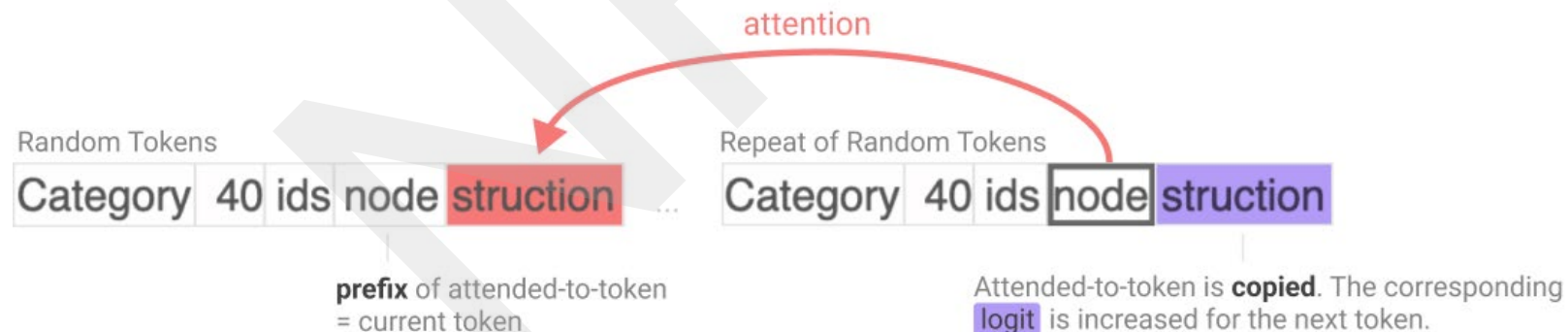


# Induction head: revisiting the formal definition

Formally, we define an induction head as one which exhibits the following two properties on a repeated random sequence of tokens:

- Prefix matching: The head attends back to previous tokens that were followed by the current and/or recent tokens. That is, it attends to the token which induction would suggest comes next.
- Copying: The head's output increases the logit corresponding to the attended-to token.

In other words, induction heads are any heads that empirically increase the likelihood of **[B]** given **[A] [B] . . . [A]** when shown a repeated sequence of completely random tokens. An illustration of induction heads' behavior is shown here:



# In-context Learning and Induction Heads

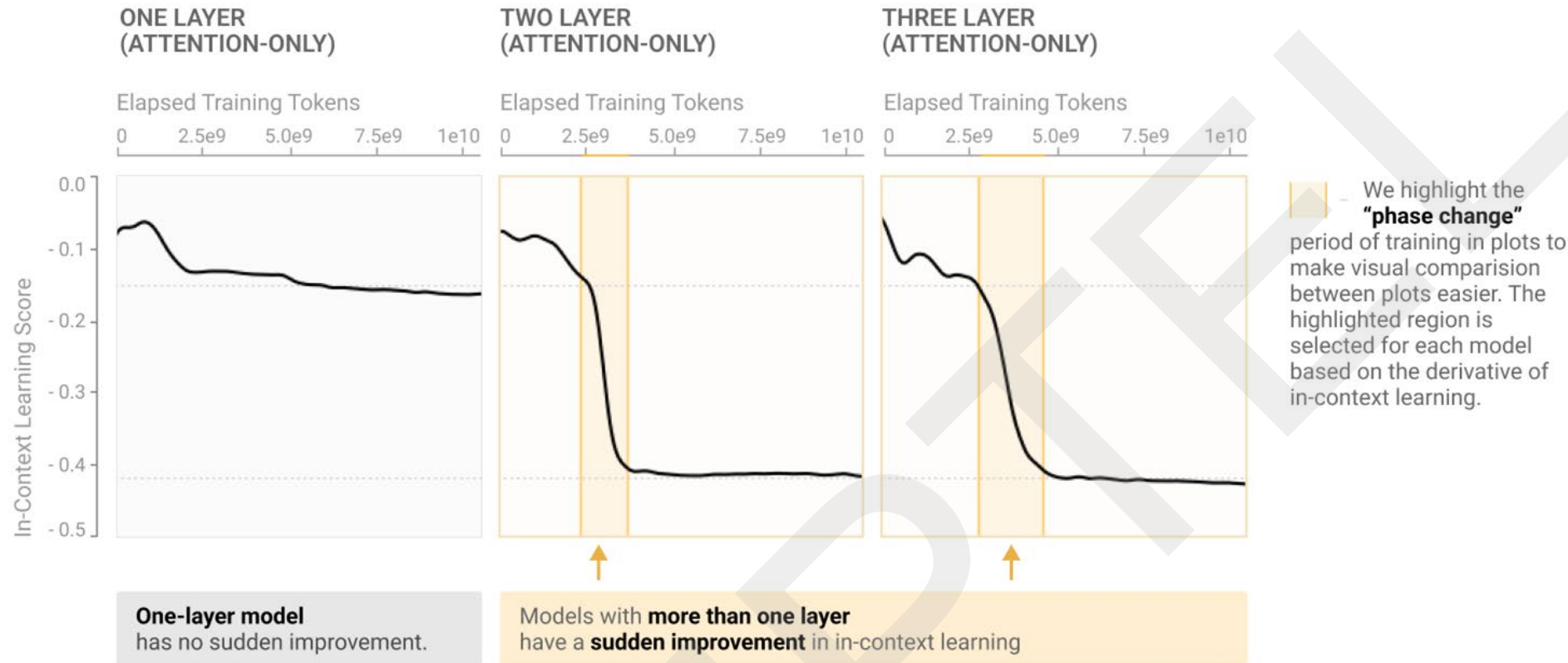
**Argument 1** (Macroscopic co-occurrence): Transformer language models undergo a “phase change” early in training, during which induction heads form and simultaneously in-context learning improves dramatically.

**Argument 2** (Macroscopic co-perturbation): When we change the transformer architecture in a way that shifts whether induction heads can form (and when), the dramatic improvement in in-context learning shifts in a precisely matching way.

**Argument 3** (Direct ablation): When we directly “knock out” induction heads at test-time in small models, the amount of in-context learning greatly decreases.

**Argument 4** (Specific examples of induction head generality): Although we define induction heads very narrowly in terms of copying literal sequences, we empirically observe that these same heads also appear to implement more sophisticated types of in-context learning, including highly abstract behaviors, making it plausible they explain a large fraction of in-context learning.

## MODELS WITH MORE THAN ONE LAYER HAVE AN ABRUPT IMPROVEMENT IN IN-CONTEXT LEARNING

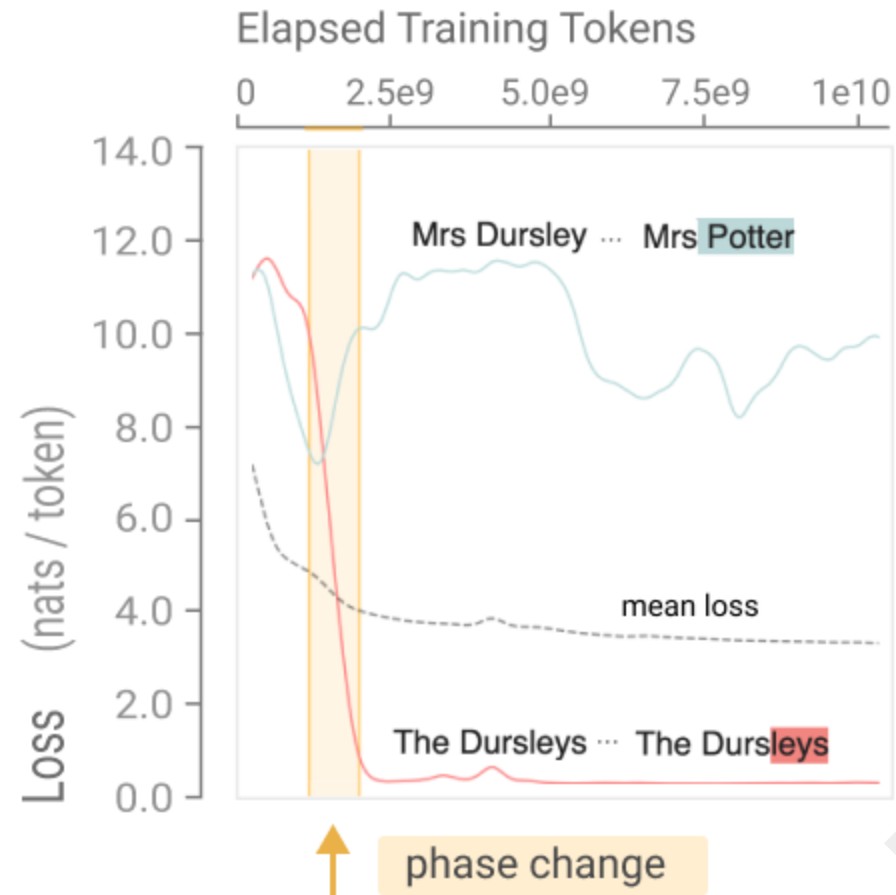


In-context learning score: the loss of the 500th token in the context minus the average loss of the 50th token in the context, averaged over dataset examples.

Why this choice? The 500th token is near the end of a length-512 context, and the 50th token is far enough into the context that some basic properties of the text have been established (such as language and document type) while still being near the start.

## PER-TOKEN LOSSES OVER TRAINING

We look at two specific tokens in the Harry Potter passage a to highlight their movement during the phase change.



**Mrs Potter** : This token's loss increases during the phase change, because the model incorrectly predicts that the next token should be "D" from "Dursley".

**The Dursleys** : This token's loss decreases during the phase change, because the model correctly predicts "leys" based on a previous example.

# In-context Learning and Induction Heads

**Argument 5** (Mechanistic plausibility of induction head generality): For small models, we can explain mechanistically how induction heads work, and can show they contribute to in-context learning. Furthermore, the actual mechanism of operation suggests natural ways in which it could be repurposed to perform more general in-context learning.

**Argument 6** (Continuity from small to large models): In the previous arguments, the case for induction heads explaining in-context learning is stronger for small models than for large ones. However, many behaviors and data related to both induction heads and in-context learning are smoothly continuous from small to large models, suggesting the simplest explanation is that mechanisms are the same.

# Recap: Zero-shot vs Few-shot Prompts

## Zero-shot prompt

Prompting without examples

Classify the text into neutral, negative, or positive.

Text: *I think the food was okay.*

Sentiment: ...

## One-shot prompt

Prompting with a single example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**

Text: *I think the food was okay.*

Sentiment:

## Few-shot prompt

Prompting with more than one example

Classify the text into neutral, negative, or positive.

Text: I think the food was alright.

Sentiment: **Neutral**.

Text: I think the food was great!

Sentiment: **Positive**.

Text: I think the food was horrible...

Sentiment: **Negative**.

Text: *I think the food was okay.*

Sentiment:

## Interesting questions:

- Which examples to choose?
- Does the order matter?
- Are all labels needed?

....

# Few-shot prompts: order sensitivity

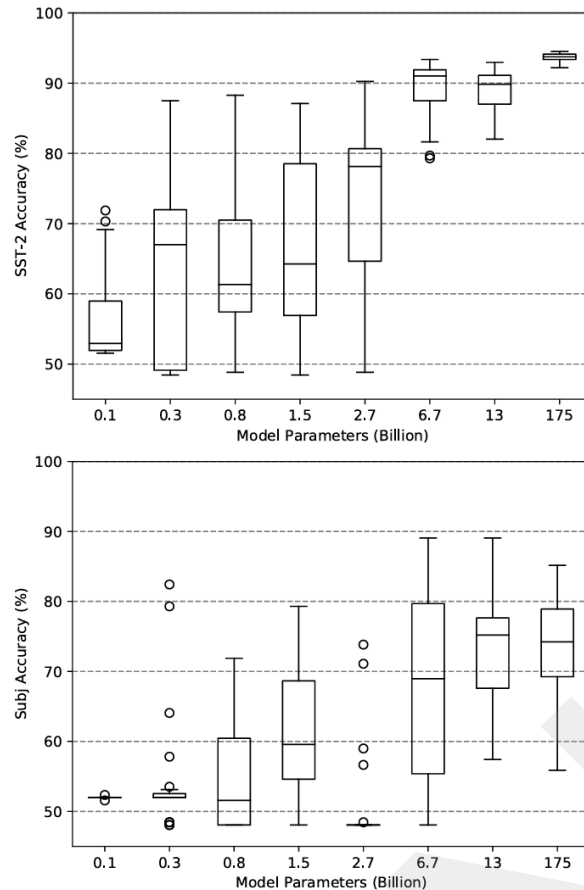
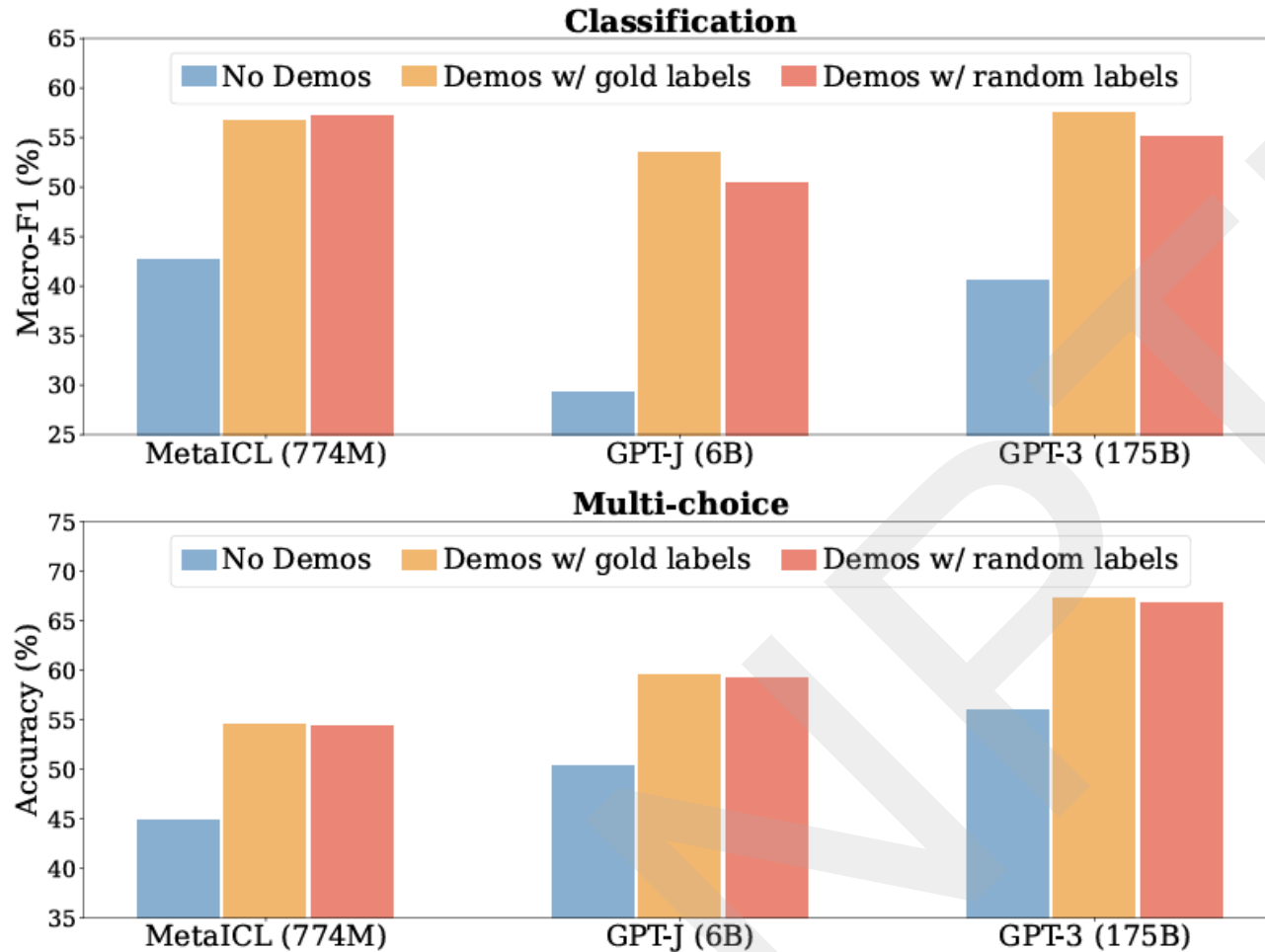


Figure 1: Four-shot performance for 24 different sample orders across different sizes of GPT-family models (GPT-2 and GPT-3) for the SST-2 and Subj datasets.

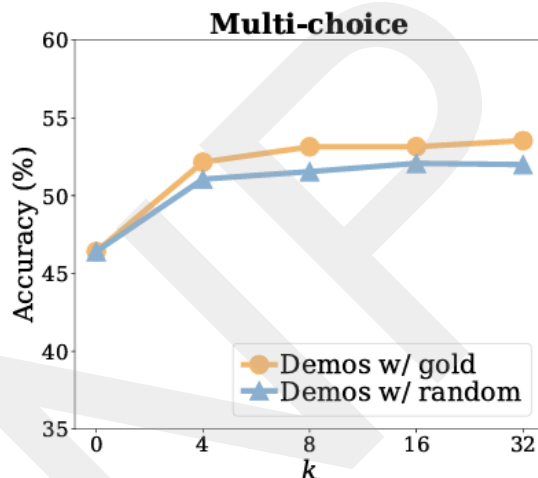
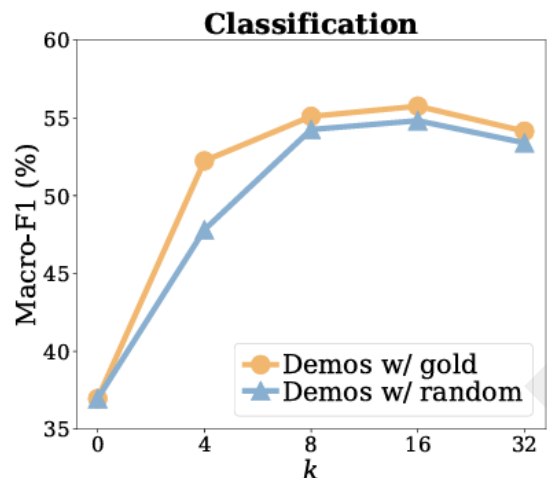
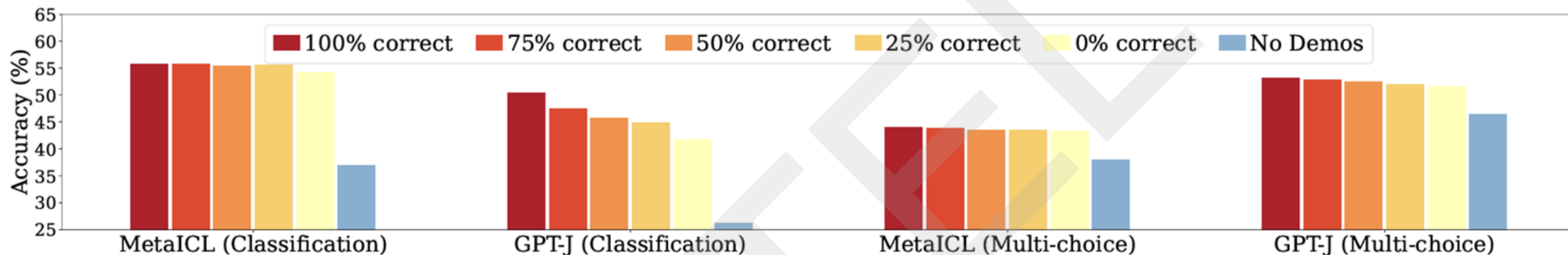
- Few-shot prompts suffer from order sensitivity, in that for the same prompt **the order in which samples are provided can make the difference** between state-of-the-art and random performance.
- The paper finds that this is **present across tasks, model sizes, prompt templates, samples, and number of training samples.**

# Some very counterintuitive effects



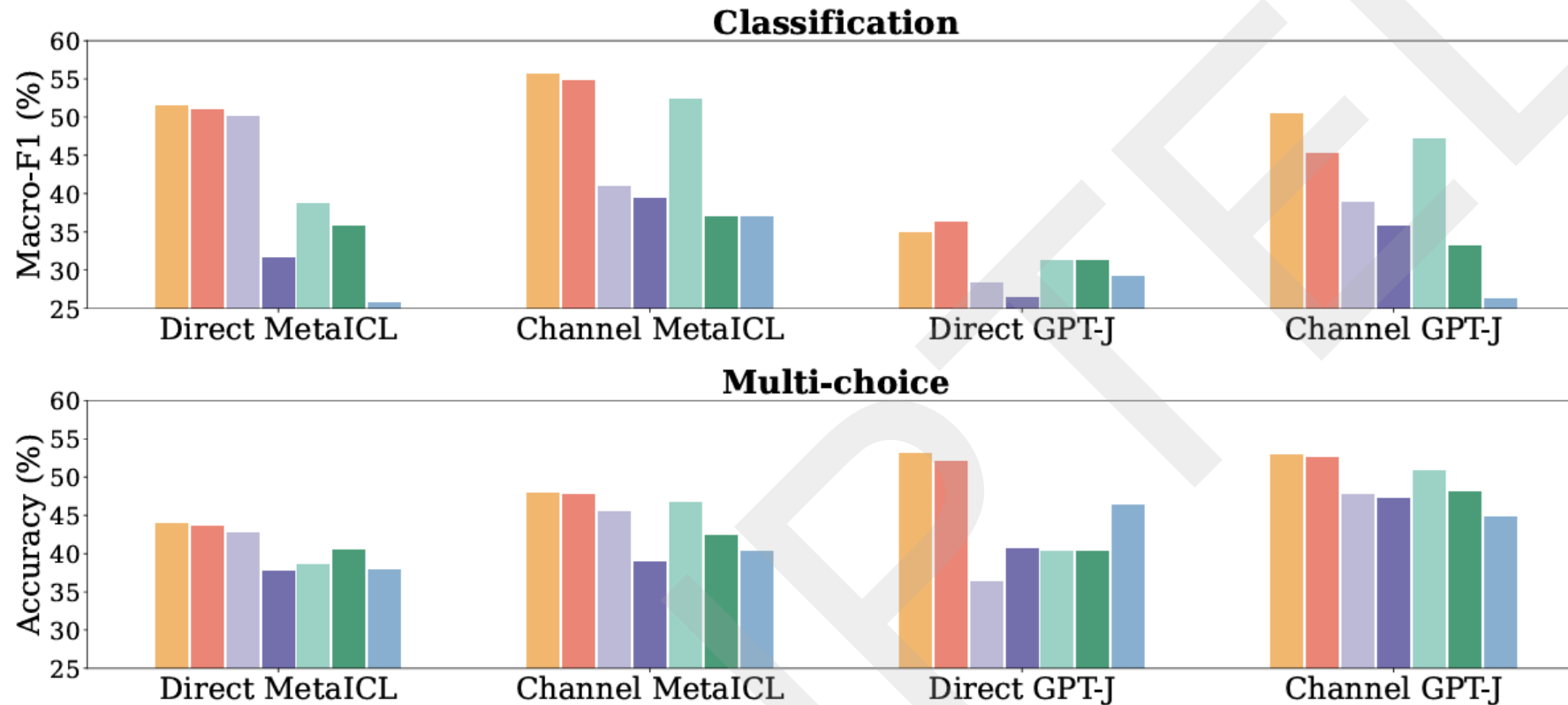
Results in classification (top) and multichoice tasks (bottom), using three LMs with varying size. In-context learning ***performance drops only marginally when labels in the demonstrations are replaced by random labels.***

# Some very counterintuitive effects



Ablations on varying numbers of examples in the demonstrations ( $k$ )

# Why does in-context learning work?



|                      | F | L | I | M |
|----------------------|---|---|---|---|
| Gold labels          | ✓ | ✓ | ✓ | ✓ |
| Random labels        | ✓ | ✓ | ✓ | ✗ |
| OOD + Random labels  | ✓ | ✓ | ✗ | ✗ |
| Random labels only   | ✗ | ✓ | ✗ | ✗ |
| Random English words | ✓ | ✗ | ✓ | ✗ |
| No labels            | ✗ | ✗ | ✓ | ✗ |
| No demonstrations    | ✗ | ✗ | ✗ | ✗ |

*F: Format*

*L: Label space*

*I: Input distribution*

*M: Input-Label Mapping*

The format is likely easier to exploit than the input-label mapping

# REFERENCES

A Mathematical Framework for Transformer Circuits

<https://transformer-circuits.pub/2021/framework/index.html>

In-context Learning and Induction Heads

<https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>



**THANK YOU**



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

# DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 43 : Advanced Prompting Techniques



**PROF. PAWAN GOYAL**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

## CONCEPTS COVERED

- Few-shot prompts: How to select demonstrations?
- Chain-of-thought Prompting
- Self-consistency, Tree-of-thought, Self-ask
- Program-aided prompts

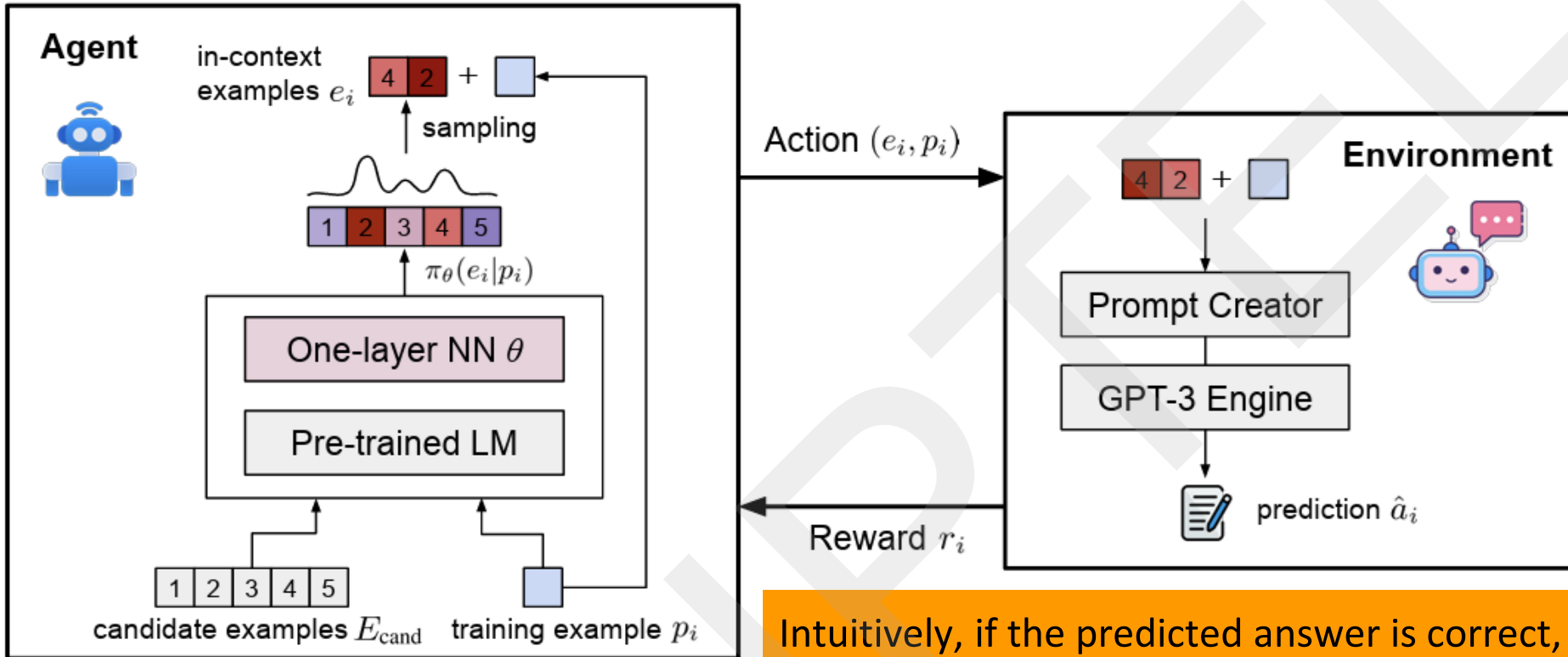
# Few-shot prompt: How to select demonstrations?



- Using demonstrations that are similar to the current input seems to improve performance.
- It can thus be useful to dynamically retrieve demonstrations for each input, based on their similarity to the current example
- for example, comparing the embedding of the current example with embeddings of each of the training set example to find the best top-T

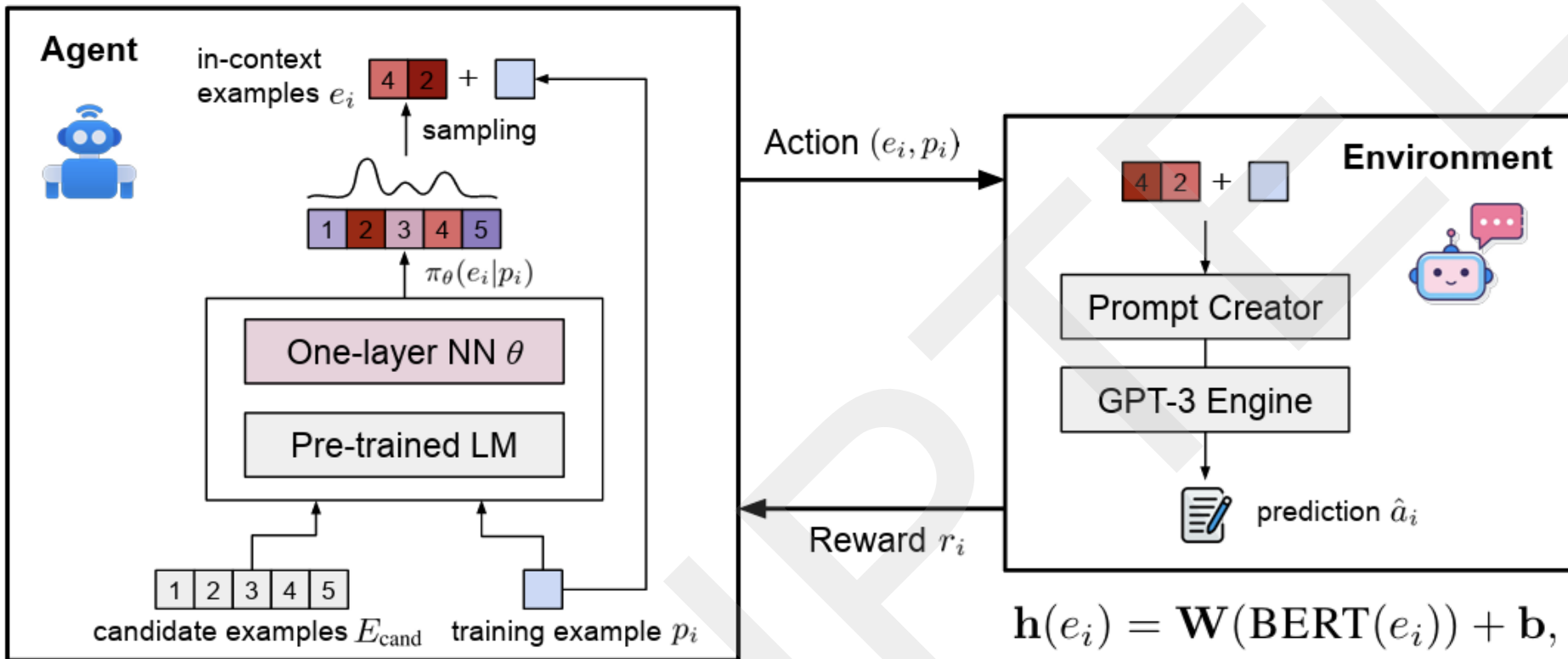
You may even learn how to select these dynamically. How?

# Few-shot prompt: How to select demonstrations?



Intuitively, if the predicted answer is correct, we update the policy so that the probability of selecting the same prompts gets higher. Otherwise, we update the policy to reduce the probability of selecting such less matched examples.

# Few-shot prompt: How to select demonstrations?



Pretrained embeddings are frozen. Only the one-layer NN is trained via policy gradient

$$\mathbf{h}(e_i) = \mathbf{W}(\text{BERT}(e_i)) + \mathbf{b},$$

$$\mathbf{h}(p_i) = \mathbf{W}(\text{BERT}(p_i)) + \mathbf{b},$$

$$\pi_\theta(e_i|p_i) = \frac{\exp[\mathbf{h}(e_i) \cdot \mathbf{h}(p_i)]}{\sum_{e'_i \in E_{\text{cand}}} \exp[\mathbf{h}(e'_i) \cdot \mathbf{h}(p_i)]}.$$

# Chain-of-thought (CoT) prompting

Rather than coming to a conclusion directly, the CoT methods instruct LLMs

- to generate reasoning steps or
- to learn from demonstrations of detailed reasoning processes provided in the prompts

Q: Please calculate the average of the numbers 2, 4, and 9.

A: The answer is 6.

Q: Please calculate the average of the numbers 1, 3, 5, and 7.

A: The answer is 4.

Q: Please calculate the average of the numbers 2, 4, and 9.

A: The answer is 7.

Although we have shown a similar question-answer pair, it remains difficult for the LLM to reason out the correct answer

# Chain-of-thought (CoT) prompting

In CoT, they may gain more from detailed problem-solving steps that are used to derive the answers.

To do this, we can *incorporate some reasoning steps into the prompt* to obtain a CoT prompt.

Q: Please calculate the mean square of the numbers 1, 3, 5, and 7.

A: Calculate the square of each number:  $1^2 = 1$ ,  $3^2 = 9$ ,  $5^2 = 25$ , and  $7^2 = 49$ . Sum the squares,  $1 + 9 + 25 + 49 = 84$ . There are 4 numbers in total. Divide the sum by the number of items,  $84/4 = 21$ . The answer is 21.

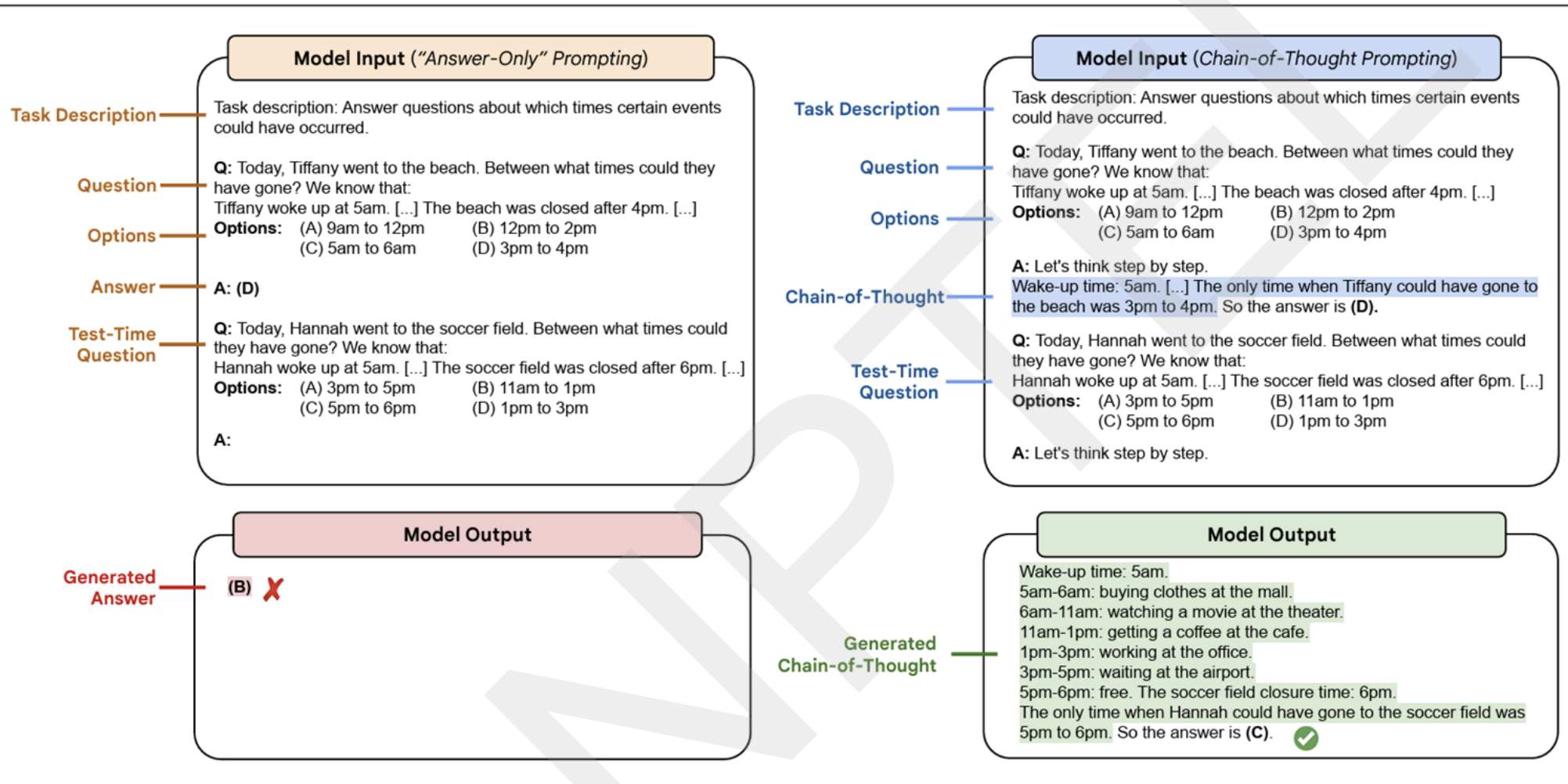
Q: Please calculate the average of the numbers 2, 4, and 9.

A: Calculate  $2 + 4 + 9$ , which equals 15. There are three numbers. Divide the total sum by the count, resulting in  $15/3 = 5$ . The answer is 5.

# CoT prompting: Advantage

- CoT allows LLMs to **decompose complex problems** into smaller, sequential reasoning steps. This somewhat mirrors human problem-solving behaviors, making it particularly effective for tasks requiring detailed, multi-step reasoning.
- CoT makes the reasoning process more transparent and interpretable. Since all reasoning steps are visible, we can understand and interpret how a conclusion was reached.
- If users can follow the logic behind the reasoning process, they will be more likely to trust the predictions of an LLM. This is particularly important when applying LLMs in fields like medicine, education, and finance.
- CoT is an in-context learning approach, and thus, it is applicable to most well-trained, off-the-shelf LLMs. Moreover, CoT provides efficient ways to adapt LLMs to different types of problems.

# CoT example: Temporal Sequencing



# Few-shot vs Zero-shot CoT

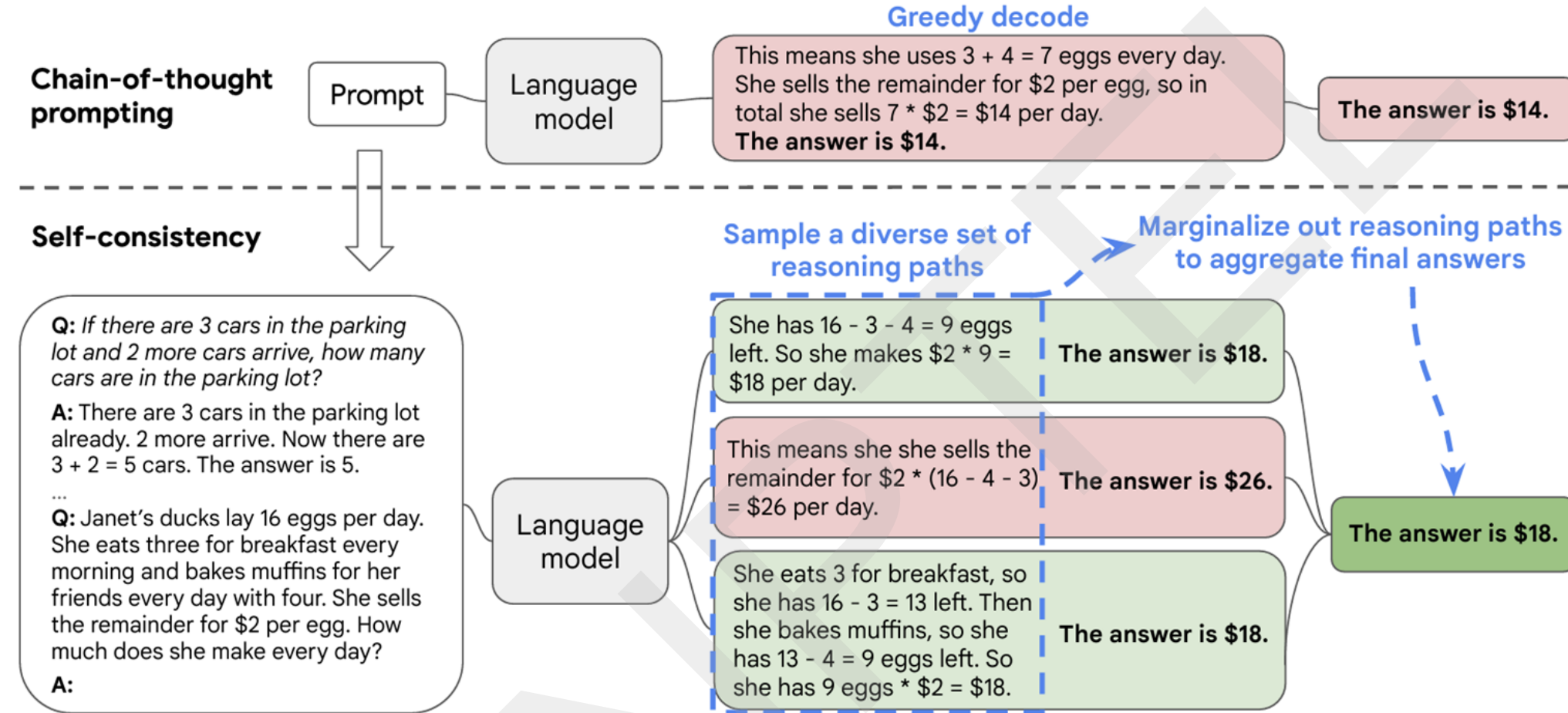
- The method described above requires providing one or more examples of CoT reasoning, typically called the few-shot CoT method.
- By contrast, the zero-shot CoT method does not require such examples. It instead prompts LLMs to reason step-by-step. For example:

Q: Please calculate the average of the numbers 2, 4, and 9.

A: Let's think step-by-step.

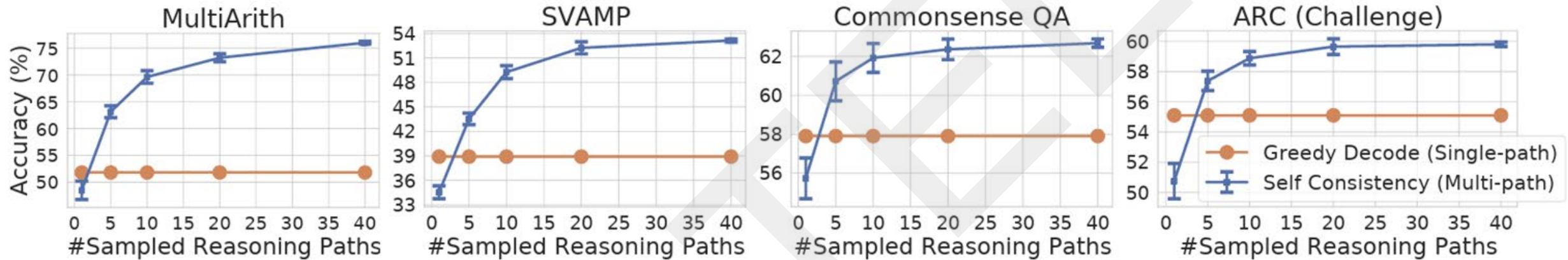
We have three numbers: 2, 4, and 9. Add these numbers together,  $2 + 4 + 9 = 15$ . Determine how many numbers there are, which in this case is three. The average is calculated by dividing the total sum by the number of elements. Completing the division gives  $15/3 = 5$ . So the answer is 5.

# Self-consistency with CoT



Self-consistency improves chain of thought reasoning in language models: <https://arxiv.org/pdf/2203.11171>

# Self-consistency with CoT



Self-consistency (blue) significantly improves accuracy over CoT-prompting with greedy decoding (orange) across arithmetic and commonsense reasoning tasks, over LaMDA-137B. Sampling **a higher number of diverse reasoning paths** consistently improves reasoning accuracy.

Self-consistency improves chain of thought reasoning in language models: <https://arxiv.org/pdf/2203.11171>

# Self-consistency with CoT vs. beam search

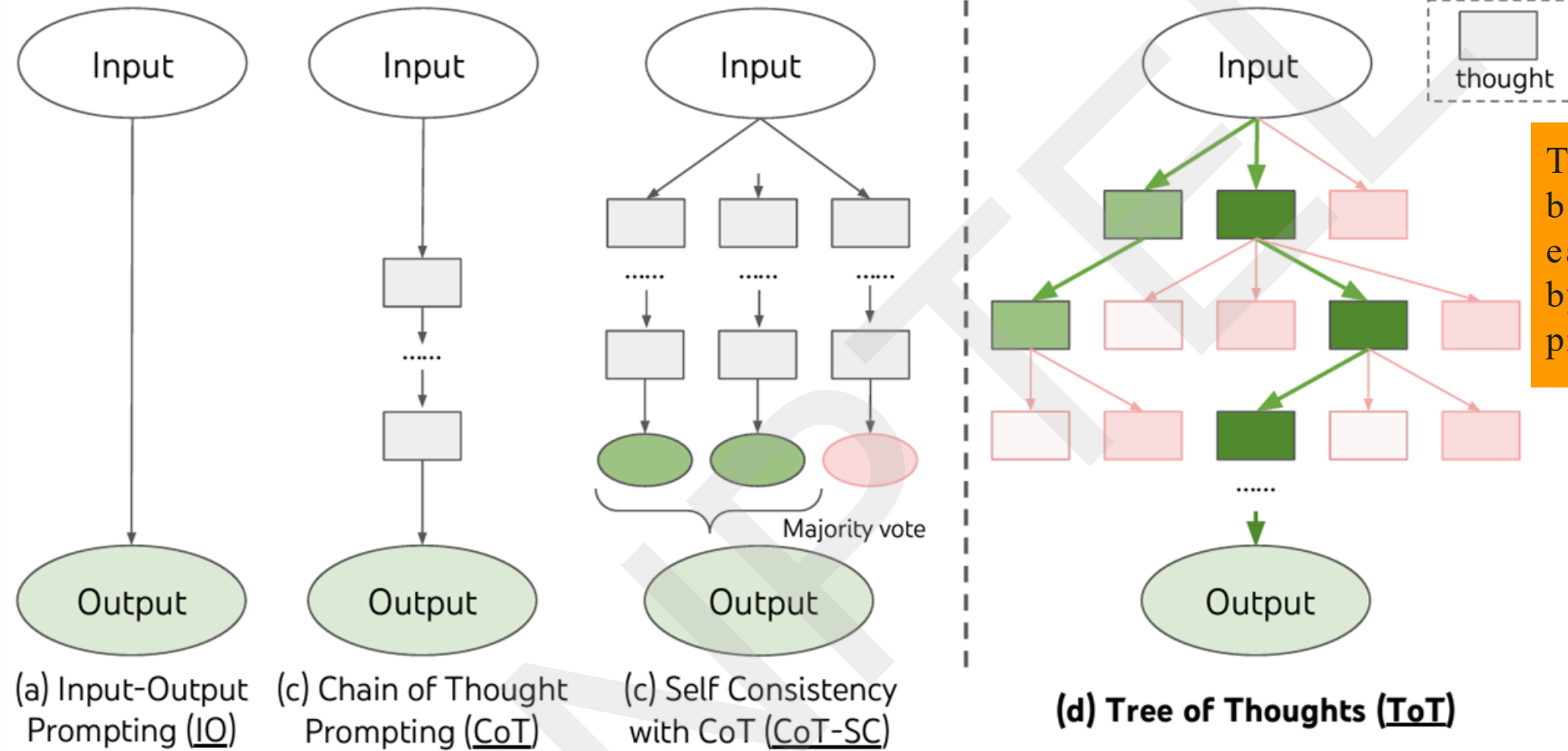


| Beam size / Self-consistency paths |                                    | 1              | 5                                | 10                               | 20                               | 40                               |
|------------------------------------|------------------------------------|----------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
| AQuA                               | Beam search decoding (top beam)    | 23.6           | 19.3                             | 16.1                             | 15.0                             | 10.2                             |
|                                    | Self-consistency using beam search | 23.6           | 19.8 $\pm$ 0.3                   | 21.2 $\pm$ 0.7                   | 24.6 $\pm$ 0.4                   | 24.2 $\pm$ 0.5                   |
|                                    | Self-consistency using sampling    | 19.7 $\pm$ 2.5 | <b>24.9 <math>\pm</math> 2.6</b> | <b>25.3 <math>\pm</math> 1.8</b> | <b>26.7 <math>\pm</math> 1.0</b> | <b>26.9 <math>\pm</math> 0.5</b> |
| MultiArith                         | Beam search decoding (top beam)    | 10.7           | 12.0                             | 11.3                             | 11.0                             | 10.5                             |
|                                    | Self-consistency using beam search | 10.7           | 11.8 $\pm$ 0.0                   | 11.4 $\pm$ 0.1                   | 12.3 $\pm$ 0.1                   | 10.8 $\pm$ 0.1                   |
|                                    | Self-consistency using sampling    | 9.5 $\pm$ 1.2  | 11.3 $\pm$ 1.2                   | <b>12.3 <math>\pm</math> 0.8</b> | <b>13.7 <math>\pm</math> 0.9</b> | <b>14.7 <math>\pm</math> 0.3</b> |

Beam search yields a lower diversity in the outputs, while in self-consistency the diversity of the reasoning paths is the key to a better performance.

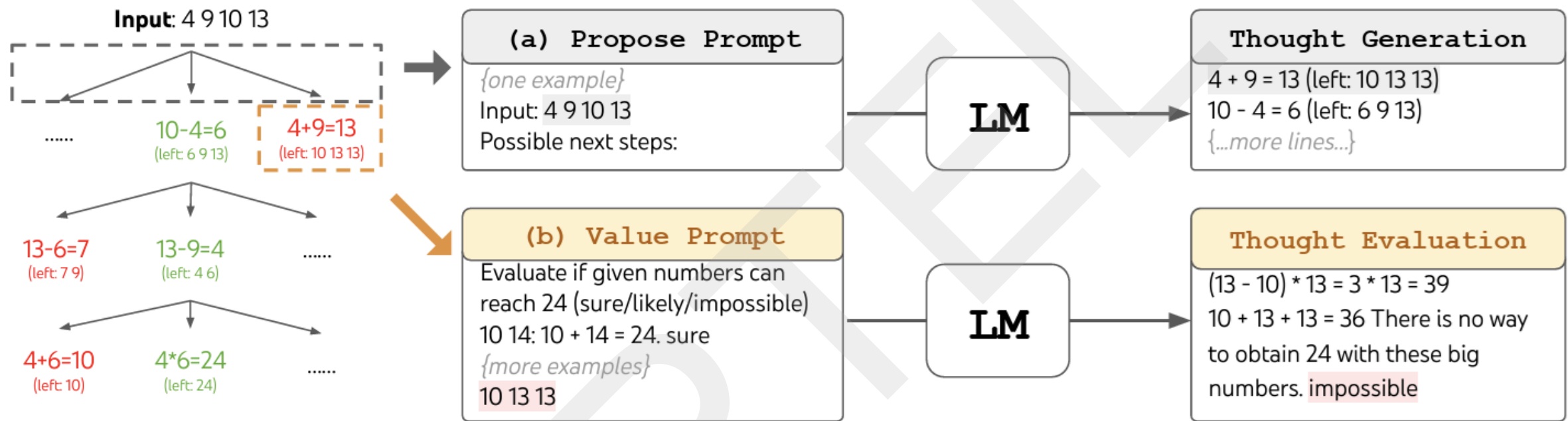
Self-consistency improves chain of thought reasoning in language models: <https://arxiv.org/pdf/2203.11171>

# Tree-of-thought (ToT)



The search process can be BFS or DFS while each state is evaluated by a classifier (via a prompt) or majority vote.

# Tree-of-thought (ToT)



*Game of 24 is a mathematical reasoning challenge, where the goal is to use 4 numbers and basic arithmetic operations (+-\*/ ) to obtain 24.*

# Self-Ask

## Direct Prompting

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Answer: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?

Answer: Franklin D. Roosevelt



## Chain of Thought

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Answer: Theodor Haecker was 65 years old when he died. Harry Vaughan Watkins was 69 years old when he died.

So the final answer (the name of the person) is: Harry Vaughan Watkins.

Question: Who was president of the U.S. when superconductivity was discovered?

Answer: Superconductivity was discovered in 1911 by Heike Kamerlingh Onnes. Woodrow Wilson was president of the United States from 1913 to 1921. So the final answer (the name of the president) is: Woodrow Wilson.



## Self-Ask

GPT-3

Question: Who lived longer, Theodor Haecker or Harry Vaughan Watkins?

Are follow up questions needed here: Yes.

Follow up: How old was Theodor Haecker when he died?

Intermediate answer: Theodor Haecker was 65 years old when he died.

Follow up: How old was Harry Vaughan Watkins when he died?

Intermediate answer: Harry Vaughan Watkins was 69 years old when he died.

So the final answer is: Harry Vaughan Watkins

Question: Who was president of the U.S. when superconductivity was discovered?

Are follow up questions needed here: Yes.

Follow up: When was superconductivity discovered?

Intermediate answer: Superconductivity was discovered in 1911.

Follow up: Who was president of the U.S. in 1911?

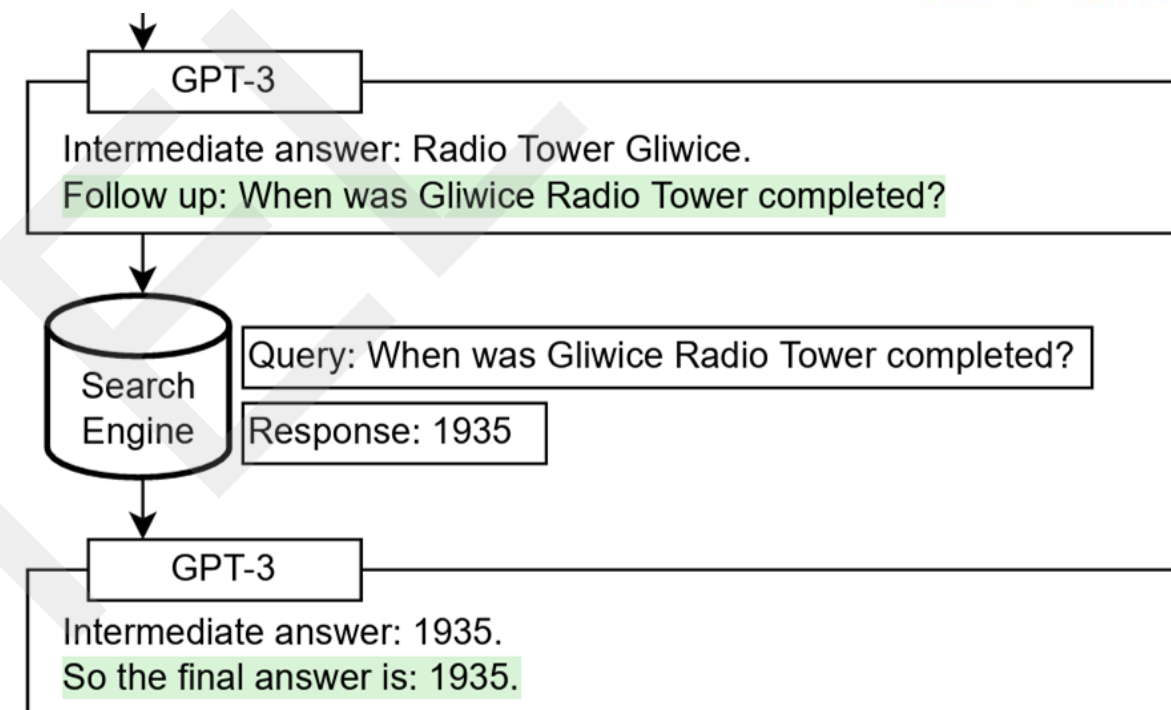
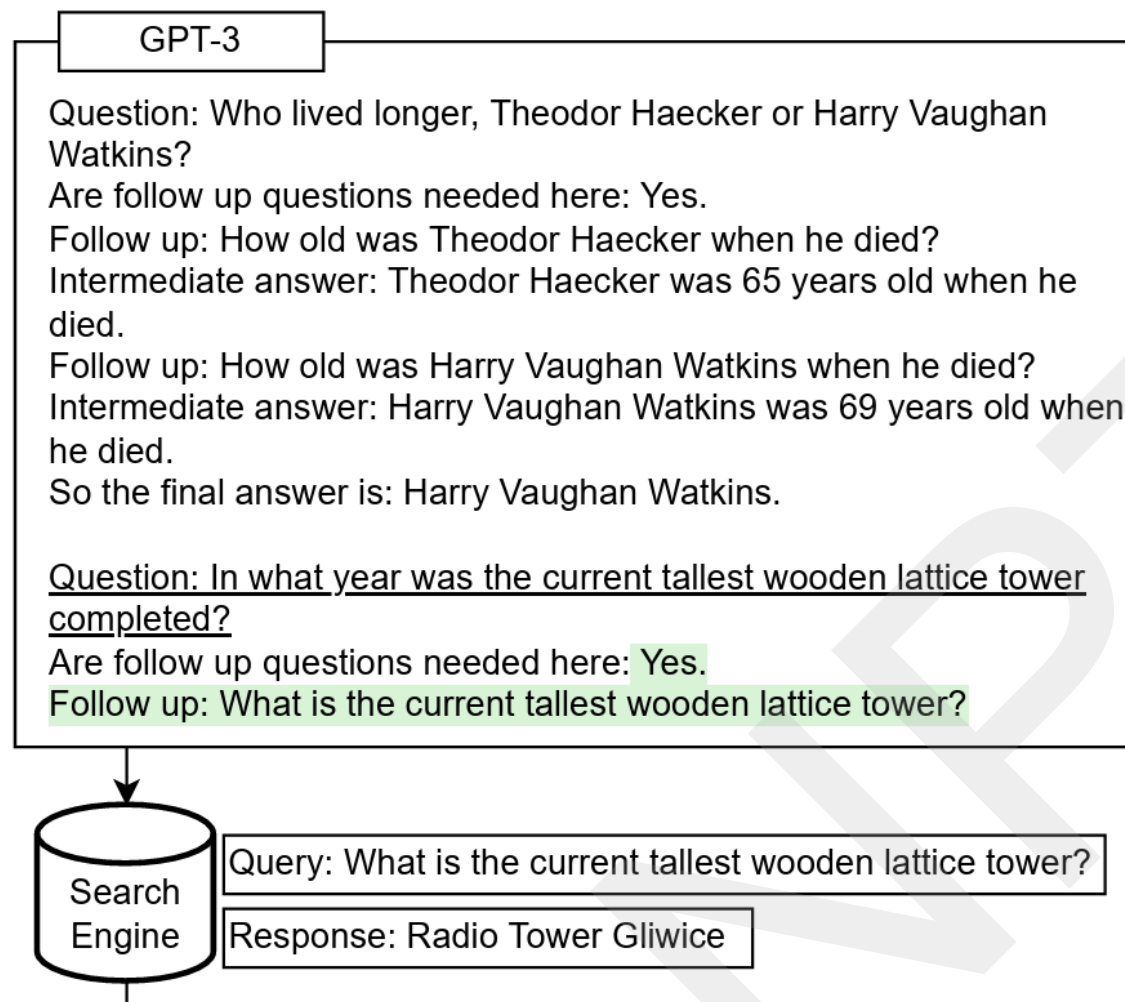
Intermediate answer: William Howard Taft.

So the final answer is: William Howard Taft.



Text with a white background is the prompt, text with a green background is the LM output, and underlined text is the inference-time question.

# Self-Ask + Search Engine



# Program-aided Language Models (PAL)

## Chain-of-Thought (Wei et al., 2022)

### Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls. 2 cans of 3 tennis balls each is 6 tennis balls.  $5 + 6 = 11$ . The answer is 11.

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

### Model Output

A: The bakers started with 200 loaves. They sold 93 in the morning and 39 in the afternoon. So they sold  $93 + 39 = 132$  loaves. The grocery store returned 6 loaves. So they had  $200 - 132 - 6 = 62$  loaves left. The answer is 62.



## Program-aided Language models (this work)

### Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 tennis balls.

`tennis_balls = 5`

2 cans of 3 tennis balls each is

`bought_balls = 2 * 3`

tennis balls. The answer is

`answer = tennis_balls + bought_balls`

Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves of bread did they have left?

### Model Output

A: The bakers started with 200 loaves

`loaves_baked = 200`

They sold 93 in the morning and 39 in the afternoon

`loaves_sold_morning = 93`

`loaves_sold_afternoon = 39`

The grocery store returned 6 loaves.

`loaves_returned = 6`

The answer is

`answer = loaves_baked - loaves_sold_morning  
- loaves_sold_afternoon + loaves_returned`

`>>> print(answer)`

74



Program-aided Language models (PAL) generate intermediate steps and Python code. This shifts the role of running the reasoning steps from the language model to the Python interpreter.

# Program-of-Thought (PoT) Prompting

Question: In Fibonacci sequence, it follows the rule that each number is equal to the sum of the preceding two numbers. Assuming the first two numbers are 0 and 1, what is the 50th number in Fibonacci sequence?

The first number is 0, the second number is 1, therefore, the third number is  $0+1=1$ . The fourth number is  $1+1=2$ . The fifth number is  $1+2=3$ . The sixth number is  $2+3=5$ . The seventh number is  $3+5=8$ . The eighth number is  $5+8=13$ .  
..... (Skip 1000 tokens)  
The 50th number is 32,432,268,459.

CoT

```
length_of_fibonacci_sequence = 50
fibonacci_sequence = np.zeros(length_of_)
fibonacci_sequence[0] = 0
fibonacci_sequence[1] = 1
For i in range(3, length_of_fibonacci_sequence):
    fibonacci_sequence[i] = fibonacci_sequence[i-1] +
    fibonacci_sequence[i-2]
ans = fibonacci_sequence[-1]
```

PoT



- PoT uses language models (mainly Codex) to generate text and programming language statements, and finally an answer.
- In PoT, the computation can be delegated to a program interpreter, thus decoupling complex computation from reasoning and language understanding.

# Program-of-Thought (PoT) Prompting

Question: Ketty saves 20000 dollars to the bank. After three years, the sum with compound interest rate is 1000 dollars more than the sum with simple interest rate. What is the interest rate of the bank?

Assuming the interest rate is  $x$ . The sum after two years with simple interest rate is  $20000 + x * 20000 * 3 = 20000 + 60000x$ . The sum after two years with compound interest rate is  $20000 * (1 + x)^3 = 200000 + 60000 * x + 60000x^2 + 20000x^3$ . The difference can be written as  $60000x^2 + 20000x^3 = 1000$ . In order to solve  $x$ , we can use the quadratic formula.  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ , ...,  $x = \frac{-20000 \pm 6160}{120000}$ ,  $x = -0.051333$ . CoT

```
interest_rate = Symbol('x')
sum_in_two_years_with_simple_interest= 20000 +
interest_rate * 20000 * 3
sum_in_two_years_with_compound_interest = 20000 * (1 +
interest_rate)**3
# Since compound interest is 1000 more than simple interest.
ans = solve(sum_after_in_yeras_with_compound_interest -
sum_after_two_years_in_compound_interest - 1000,
interest_rate)
```

PoT

↓  
-0.051333



↓  
 $x = 0.24814$



*Can we also  
augment other  
tools?*

- PoT uses language models (mainly Codex) to generate text and programming language statements, and finally an answer.
- In PoT, the computation can be delegated to a program interpreter, thus decoupling complex computation from reasoning and language understanding.

# REFERENCES

■ Daniel Jurafsky and James H. Martin. 2024. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released January 12, 2025. [Chapter 12]

■ Hands-on Large Language Models  
■ Foundations of Large Language Models



**THANK YOU**



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

# DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 44 : Tool-aided Language Models



**PROF. PAWAN GOYAL**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

## CONCEPTS COVERED

- Why tools? ✓
- Tools-aided language models, Toolformer ↻

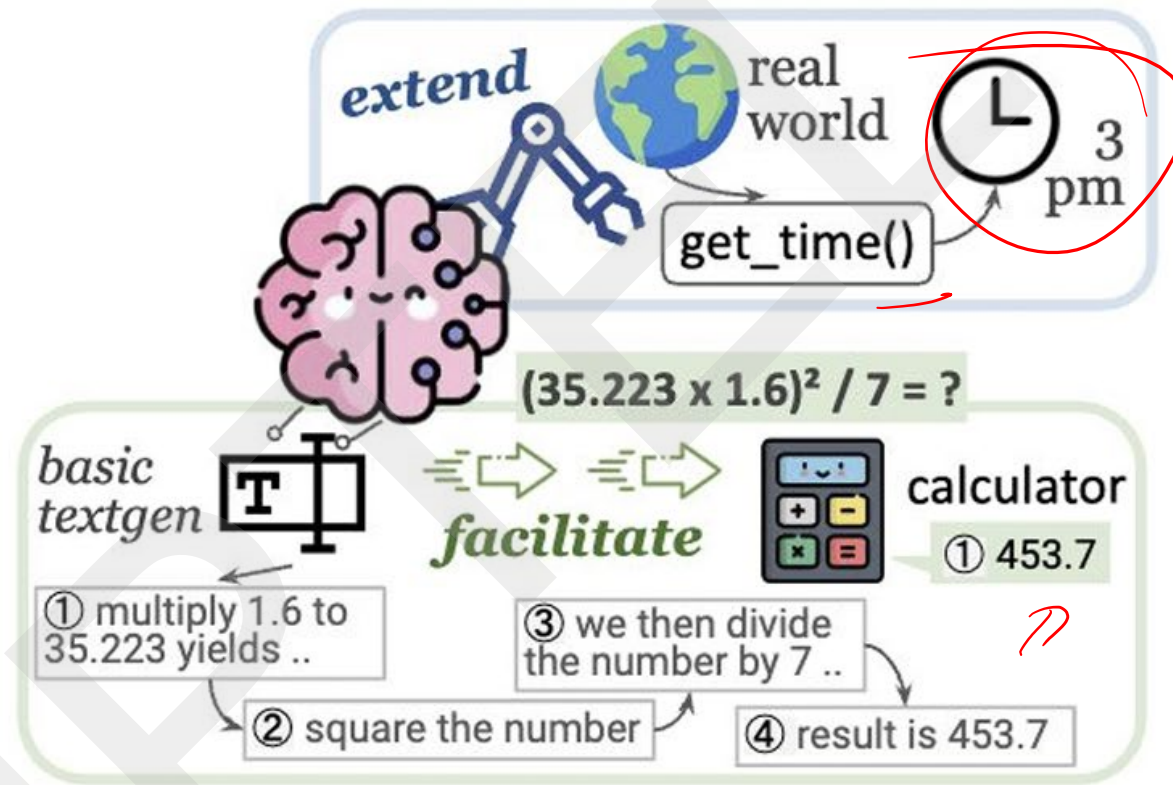
# LMs are powerful for text generation tasks. But ...

- Complex reasoning?

**Struggle**

- Access real-world information?

**Fundamentally  
unable**



# The Basic Tool Use Paradigm

Tool Use: switching between

- text-generation mode
- tool-execution mode

Tool Learning:

- inference-time prompting
- learning by training

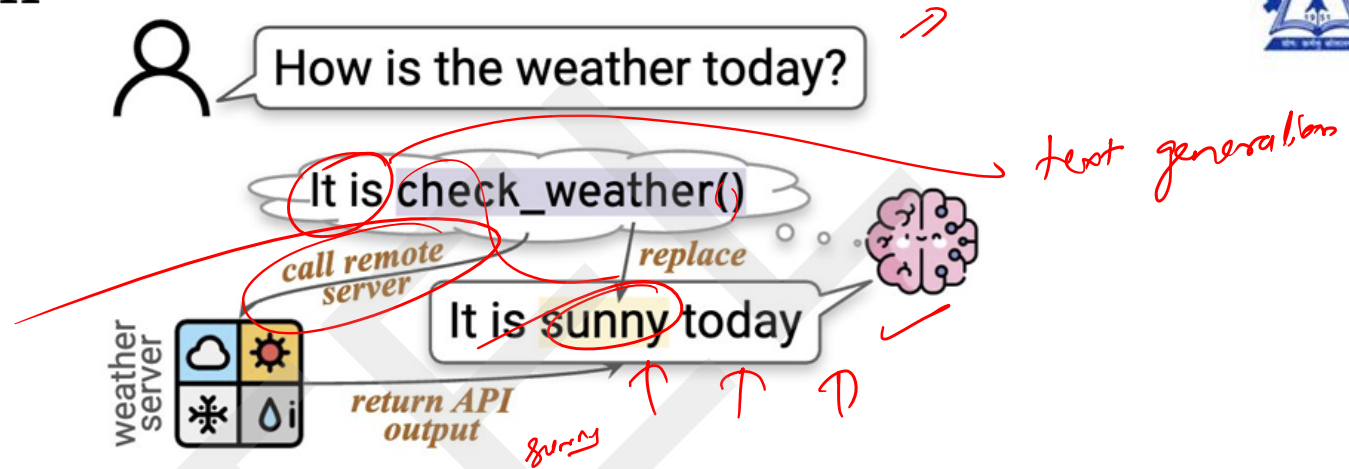


Figure 2: The basic tool use paradigm. LM calls `check_weather` tool by generating text tokens. This call triggers the server to execute the call and return the output `sunny`, using which the LM replaces the API call tokens in the response to the user.

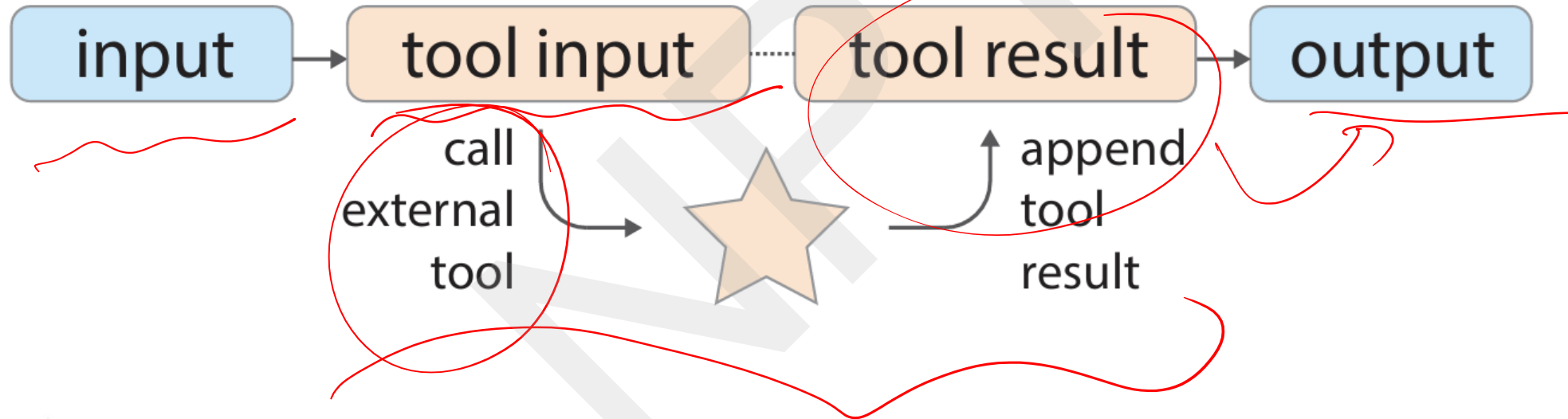
*An LM-used tool is a function interface to a computer program that runs external to the LM, where the LM generates the function calls and input arguments in order to use the tool.*

# Tool-augmented Language Model (TALM)

## Language Model



## Tool Augmented Language Model



# TALM text-to-text interface

## An abstract task:

task input text | *parameters / arguments* **tool-call** tool input text | **result** tool output text | **output** task output text

## A weather task:

how hot will it get in NYC today? | **weather** lookup region=NYC | **result** precipitation chance: 10, high temp: 20c, low-temp: 12c | **output** today's high will be 20C

LLM

[RAG]

TALM first generates a tool input conditioned on the task input text and invokes a tool's API by generating a delimiter, such as "result". Whenever this delimiter is detected, the tool API is called and its result appended to the text sequence. TALM then continues to generate the final task output.

# TALM text-to-text interface

**Question:** If Lily's test scores are 85 , 88 and 95 out of 100 in 3 different subjects , what will be her average score?

**Formula:**  $\text{Divide}(\text{Add}(85, \text{Add}(88, 95)), 3)$

**Answer:** 89.33

Example from MathQA, as a standard MathQA task and the corresponding tool-augmented sequence

|question If Lily's test scores are 85 , 88 and 95 out of 100 in 3 different subjects , what will be her average score? |formula  $\text{Divide}(\text{Add}(85, \text{Add}(88, 95)), 3)$   
|result 89.333333333333 |output 89.33

# Toolformer

The New England Journal of Medicine is a registered trademark of [QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society] the MMS.

Out of 1400 participants, 400 (or [Calculator(400 / 1400) → 0.29] 29%) passed the test.

The name derives from "la tortuga", the Spanish word for [MT("tortuga") → turtle] turtle.

The Brown Act is California's law [WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.] that requires legislative bodies, like city councils, to hold their meetings open to the public.

The model autonomously decides to call different APIs (from top to bottom)

- a question answering system
- a calculator
- a machine translation system, and
- a Wikipedia search engine

to obtain information that is useful for completing a piece of text

# Toolformer: More details

We represent each API call as a tuple  $c = (a_c, i_c)$  where  $a_c$  is the name of the API and  $i_c$  is the corresponding input. Given an API call  $c$  with a corresponding result  $r$ , we denote the linearized sequences of the API call not including and including its result, respectively, as:

$$\begin{aligned} e(c) &= \langle \text{API} \rangle a_c (i_c) \langle / \text{API} \rangle \\ e(c, r) &= \langle \text{API} \rangle a_c (i_c) \rightarrow r \langle / \text{API} \rangle \end{aligned}$$

<sup>1</sup>In practice, we use the token sequences “ [”, “]” and “→” to represent “ $\langle \text{API} \rangle$ ”, “ $\langle / \text{API} \rangle$ ” and “ $\rightarrow$ ”, respectively. This enables our approach to work without modifying the existing LM’s vocabulary. For reasons of readability, we still refer to them as “ $\langle \text{API} \rangle$ ”, “ $\langle / \text{API} \rangle$ ” and “ $\rightarrow$ ” throughout this section.

# Toolformer: How to create training data?

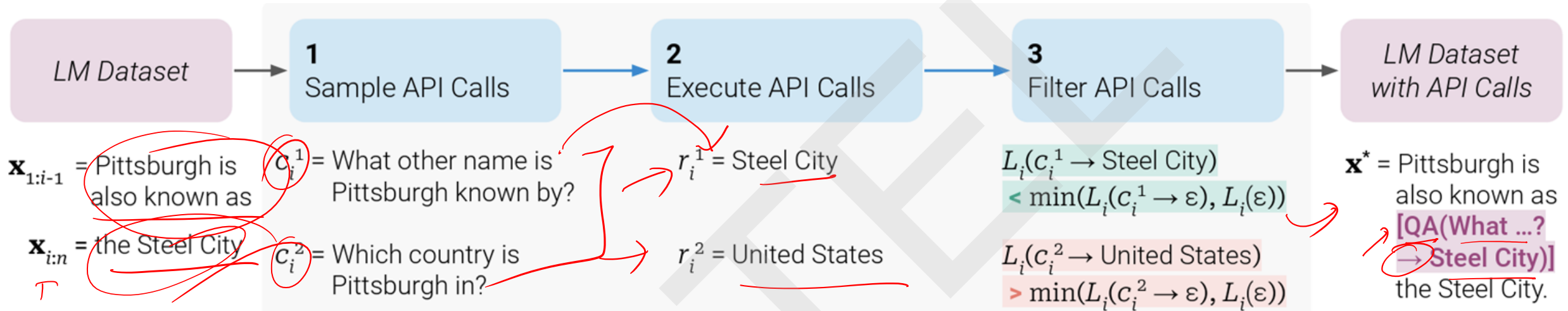


Figure 2: Key steps in our approach, illustrated for a *question answering* tool: Given an input text  $\mathbf{x}$ , we first sample a position  $i$  and corresponding API call candidates  $c_i^1, c_i^2, \dots, c_i^k$ . We then execute these API calls and filter out all calls which do not reduce the loss  $L_i$  over the next tokens. All remaining API calls are interleaved with the original text, resulting in a new text  $\mathbf{x}^*$ .

Given a dataset  $C$  of plain texts, we first convert this dataset into a dataset  $C^*$  augmented with API calls.

# Toolformer: How to create training data?

Your task is to add calls to a Question Answering API to a piece of text. The questions should help you get information required to complete the text. You can call the API by writing "[QA(question)]" where "question" is the question you want to ask. Here are some examples of API calls:

**Input:** Joe Biden was born in Scranton, Pennsylvania.

**Output:** Joe Biden was born in [QA("Where was Joe Biden born?")] Scranton, [QA("In which state is Scranton?")] Pennsylvania.

**Input:** Coca-Cola, or Coke, is a carbonated soft drink manufactured by the Coca-Cola Company.

**Output:** Coca-Cola, or [QA("What other name is Coca-Cola known by?")] Coke, is a carbonated soft drink manufactured by [QA("Who manufactures Coca-Cola?")] the Coca-Cola Company.

**Input:  $\mathbf{x}$**

**Output:**

**Sampling API Calls** For each API, we write a prompt  $P(\mathbf{x})$  that encourages the LM to annotate an example  $\mathbf{x} = x_1, \dots, x_n$  with API calls.

*And do not worry about the output, but the probability distributions to sample the API call!!*

# Toolformer: Sampling API calls

Let  $p_M(\underbrace{z_{n+1}}_{\text{token}} \mid \underbrace{z_1, \dots, z_n}_{\text{sequence}})$  be the probability that  $M$  assigns to token  $z_{n+1}$  as a continuation for the sequence  $z_1, \dots, z_n$ .

We first sample up to  $k$  candidate positions for doing API calls by computing

$$p_i = p_M(\underbrace{\langle \text{API} \rangle}_{\text{Prompt}} \mid \underbrace{P(\mathbf{x}), x_{1:i-1}}_{\text{Prompt}})$$

Keep top  $k$  positions  $I = \{i \mid p_i > \tau_s\}$   $k \approx m$

For each position, obtain upto  $m$  API calls by sampling given

$$[P(\mathbf{x}), x_1, \dots, x_{i-1}, \langle \text{API} \rangle] \longrightarrow \boxed{c_i^1, \dots, c_i^m}$$

<sup>6</sup>  
should have  $\langle \text{API} \rangle$  as an end-of-sequence token.

The response for each API call  $c_i$  needs to be a single text sequence  $r_i$

# Toolformer: Filtering API calls

Intuitively, an API call is helpful to  $M$  if providing it with both the input and the output of this call makes it easier for the model to predict future tokens, compared to

- not receiving the API call at all, or ✓
- receiving only its input. ✓

$L_i^+ = L_i(\mathbf{e}(c_i, r_i))$  only keep API calls for which

$$L_i^- = \min(L_i(\varepsilon), L_i(\mathbf{e}(c_i, \varepsilon)))$$

$$\underline{L_i^-} - \underline{L_i^+} \geq \underline{\tau_f}$$

*i.e., adding the API call and its result reduces the loss by at least a threshold, compared to not doing any API call or obtaining no result from it*

**Model  $M$  is now fine-tuned on the augmented dataset** ✓

# Toolformer: Some examples

| Example                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | $L_i^- - L_i^+$ | Useful |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|--------|
| The Flodden Window (a war memorial dedicated to The Middleton Archers), in the Grade I-listed Church of St Leonard in Middleton is said to be the oldest war memorial in the United Kingdom. <b>&lt;API&gt; WikiSearch(War memorial Flodden) → Battle of Flodden &gt; Commemoration &gt; The stained-glass Flodden Window in Middleton Parish Church [...] was constructed by Sir Richard Assheton in memory of the Battle of Flodden and the archers from Middleton who fought in it. &lt;/API&gt;</b> Sir Richard Assheton of Middleton (who built St Leonard) was granted knighthood [...] | 5.49            | ✓      |
| Note: The WL will be open on Friday, <b>&lt;API&gt; Calendar() → Today is Thursday, March 9, 2017. &lt;/API&gt;</b> <u>March 10</u> , and Sunday, March 19 for regular hours.                                                                                                                                                                                                                                                                                                                                                                                                                 | 2.11            | ✓      |
| The Nile has an approximate length of <b>&lt;API&gt; QA(What is the approximate length of the Nile?) → 6,853 km &lt;/API&gt;</b> 6,853 kilometers, the White Nile being its main source.                                                                                                                                                                                                                                                                                                                                                                                                      | 2.08            | ✓      |
| If Venus had an atmosphere similar to Earth's then you would expect Venus' mean temperature to be 499 K (1.74 x 287) rather than 735 K which is <b>&lt;API&gt; Calculator(735 / 499) → 1.47 &lt;/API&gt;</b> 1.47 (735 / 499) times hotter than it should be.                                                                                                                                                                                                                                                                                                                                 | 1.59            | ✓      |

# Toolformer: Inference

- When generating text with  $M$  after finetuning, we perform regular decoding until  $M$  produces the “→” token, indicating that it next expects the response for an API call.
- At this point, we interrupt the decoding process, call the appropriate API to get a response, and
- Continue the decoding process after inserting both the response and the `</API>` token.

| API Name            | Example Input                              | Example Output                                                                                                                                                                                                               |
|---------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Question Answering  | Where was the Knights of Columbus founded? | New Haven, Connecticut                                                                                                                                                                                                       |
| Wikipedia Search    | Fishing Reel Types                         | Spin fishing > Spin fishing is distinguished between fly fishing and bait cast fishing by the type of rod and reel used. There are two types of reels used when spin fishing, the open faced reel and the closed faced reel. |
| Calculator          | $27 + 4 * 2$                               | 35                                                                                                                                                                                                                           |
| Calendar            | $\epsilon$                                 | Today is Monday, January 30, 2023.                                                                                                                                                                                           |
| Machine Translation | sûreté nucléaire                           | nuclear safety                                                                                                                                                                                                               |

# Many recent works



## TOOLLLM: FACILITATING LARGE LANGUAGE MODELS TO MASTER 16000+ REAL-WORLD APIS

Yujia Qin<sup>1\*</sup>, Shihao Liang<sup>1\*</sup>, Yining Ye<sup>1</sup>, Kunlun Zhu<sup>1</sup>, Lan Yan<sup>1</sup>, Yaxi Lu<sup>1</sup>, Yankai Lin<sup>3†</sup>, Xin Cong<sup>1</sup>, Xiangru Tang<sup>4</sup>, Bill Qian<sup>4</sup>, Sihan Zhao<sup>1</sup>, Lauren Hong<sup>1</sup>, Runchu Tian<sup>1</sup>, Ruobing Xie<sup>5</sup>, Jie Zhou<sup>5</sup>, Mark Gerstein<sup>4</sup>, Dahai Li<sup>2,6</sup>, Zhiyuan Liu<sup>1†</sup>, Maosong Sun<sup>1†</sup>

<sup>1</sup>Tsinghua University <sup>2</sup>ModelBest Inc. <sup>3</sup>Renmin University of China

<sup>4</sup>Yale University <sup>5</sup>WeChat AI, Tencent Inc. <sup>6</sup>Zhihu Inc.

yujiqin16@gmail.com

2

## Gorilla: Large Language Model Connected with Massive APIs

Shishir G. Patil<sup>1\*</sup> Tianjun Zhang<sup>1,\*</sup> Xin Wang<sup>2</sup> Joseph E. Gonzalez<sup>1</sup>

<sup>1</sup>UC Berkeley <sup>2</sup>Microsoft Research  
sgp@berkeley.edu

## HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face

Yongliang Shen<sup>1,2,\*</sup>, Kaitao Song<sup>2,\*†</sup>, Xu Tan<sup>2</sup>,  
Dongsheng Li<sup>2</sup>, Weiming Lu<sup>1,†</sup>, Yueting Zhuang<sup>1,†</sup>  
Zhejiang University<sup>1</sup>, Microsoft Research Asia<sup>2</sup>

{syl, luwm, yzhuang}@zju.edu.cn, {kaitaosong, xuta, dongshli}@microsoft.com

# REFERENCES

Various papers cited in the slides



**THANK YOU**



N P T E L O N L I N E C E R T I F I C A T I O N C O U R S E S

# DEEP LEARNING FOR NATURAL LANGUAGE PROCESSING

Lecture 45 : Automatic Prompt Engineering



**PROF. PAWAN GOYAL**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

## CONCEPTS COVERED

- Discrete Prompt Search
- Prompt Tuning

# Need for Automatic Prompt Engineering

- Automated Prompt design involves using some form of search or generation for the most effective prompt template in a predefined search space.
- While more complex to implement, automated prompt development will usually outperform manual prompt engineering, as it is generally more complete in its search of parameter space.

Some possible approaches:

- Prompt paraphrasing
- Gradient-based discrete prompt search
- Prompt tuning

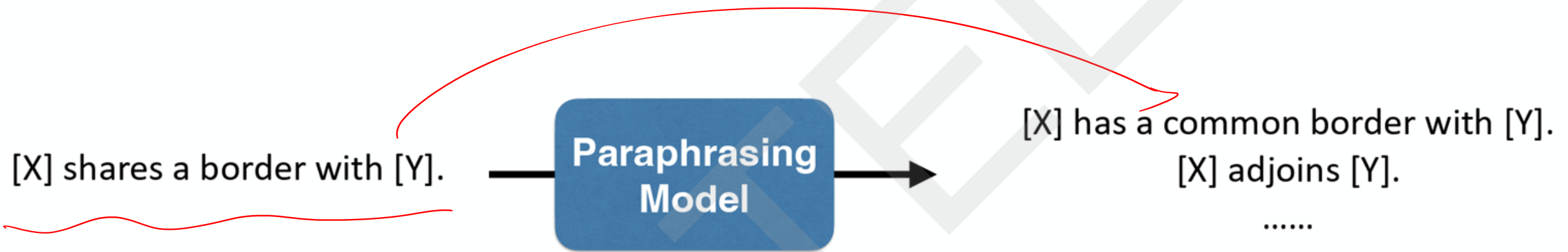
*Soft-prompts*

*token spaces*

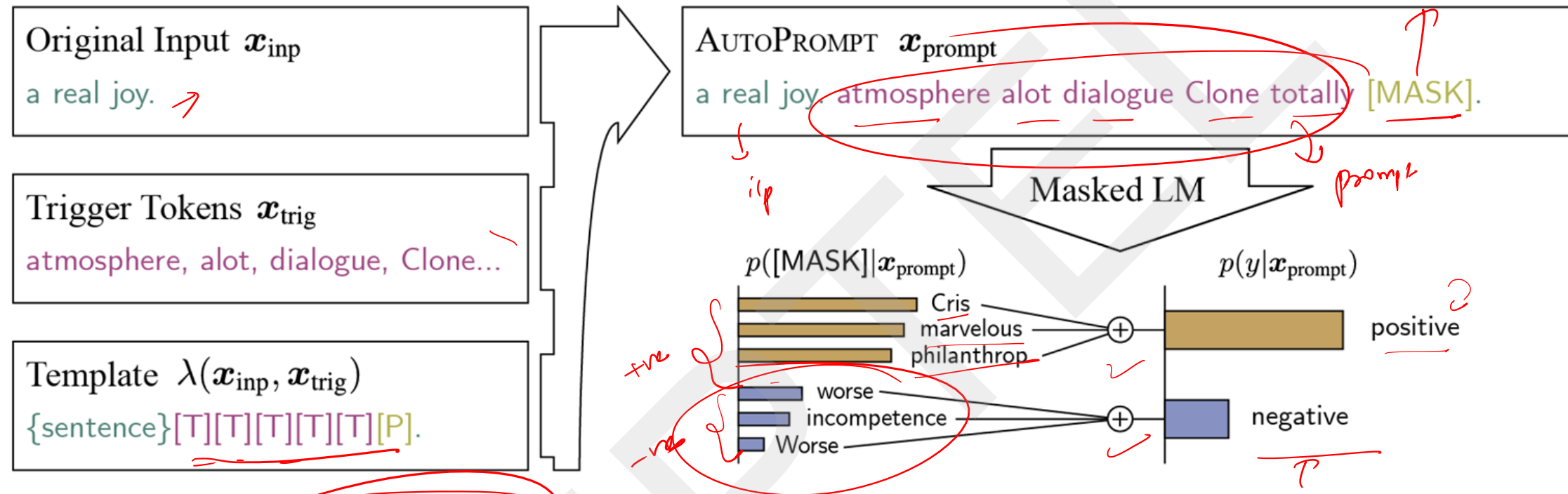
*Continuous space*

# Prompt Paraphrasing

- Paraphrase an existing prompt to get other candidates (Jiang et al. 2019)



# Gradient-based discrete prompt search



*Illustration of AUTOPROMPT applied to probe a masked language model's (MLM's) ability to perform sentiment analysis.*

# Gradient-based discrete prompt search

Template  $\lambda(\mathbf{x}_{\text{inp}}, \mathbf{x}_{\text{trig}})$   
{sentence}[T][T][T][T][T][P].

number  
hyper-parameter

Start with fixing the number of trigger tokens (e.g., 5 tokens, initialized to [MASK])

Candidate Trigger

Batch Of Examples

$p(y|x)$

[MASK]

[MASK]

[MASK]

An artful, intelligent...

0.01

The uninspiring...

0.05

It's a beautiful story...

0.03

Take a batch of examples from training set

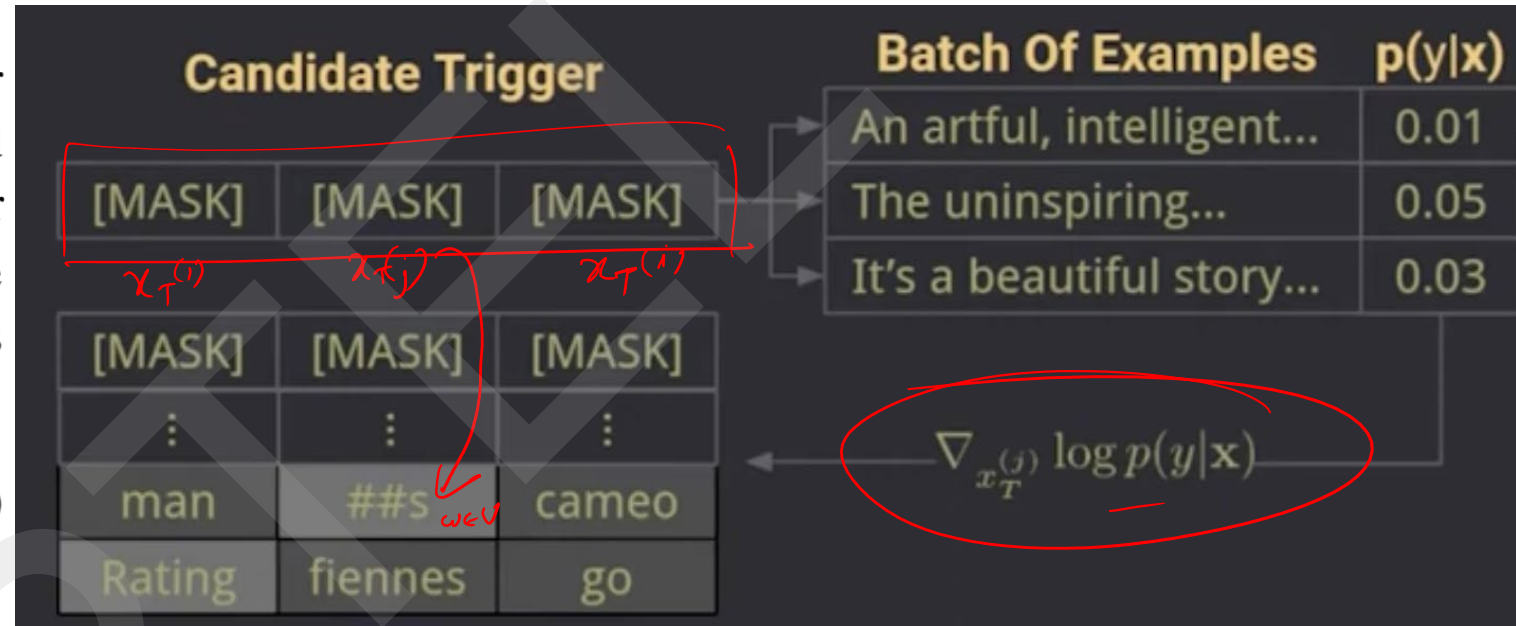
AUTOPROMPT: Eliciting Knowledge from Language Models with Automatically Generated Prompts: <https://arxiv.org/pdf/2010.15980>

# Gradient-based discrete prompt search

Formally, at each step, we compute a first-order approximation of the change in the log-likelihood that would be produced by swapping the  $j$ th trigger token  $x_{\text{trig}}^{(j)}$  with another token  $w \in \mathcal{V}$ . Then we identify a candidate set  $\mathcal{V}_{\text{cand}}$  of the top- $k$  tokens estimated to cause the greatest increase:

$$\mathcal{V}_{\text{cand}} = \underset{w \in \mathcal{V}}{\text{top-}k} \left[ \underbrace{w_{\text{in}}^T}_{\text{16b}} \nabla \log p(y|\mathbf{x}_{\text{prompt}}) \right] \quad (2)$$

where  $w_{\text{in}}$  is the input embedding of  $w$ , and the gradient is taken with respect to the input embedding of  $x_{\text{trig}}^{(j)}$ .



Why this particular form of Eq 2?

# Directional Derivatives and 1st order approx.

Let  $f$  be a differentiable function  $f : \mathbf{R}^d \rightarrow \mathbf{R}$ . We define the directional derivative of  $f$  at the point  $x \in \mathbf{R}^d$  in the direction  $v \in \mathbf{R}^d$  as

$$\nabla_v f(x) = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon v) - f(x)}{\varepsilon}.$$

This expression is easy to interpret if we drop the limit and replace equality with approximate equality. So, let's suppose that  $\varepsilon$  is very small. Then we can write

$$\frac{f(x + \varepsilon v) - f(x)}{\varepsilon} \approx \nabla_v f(x).$$

Rearranging, this implies that

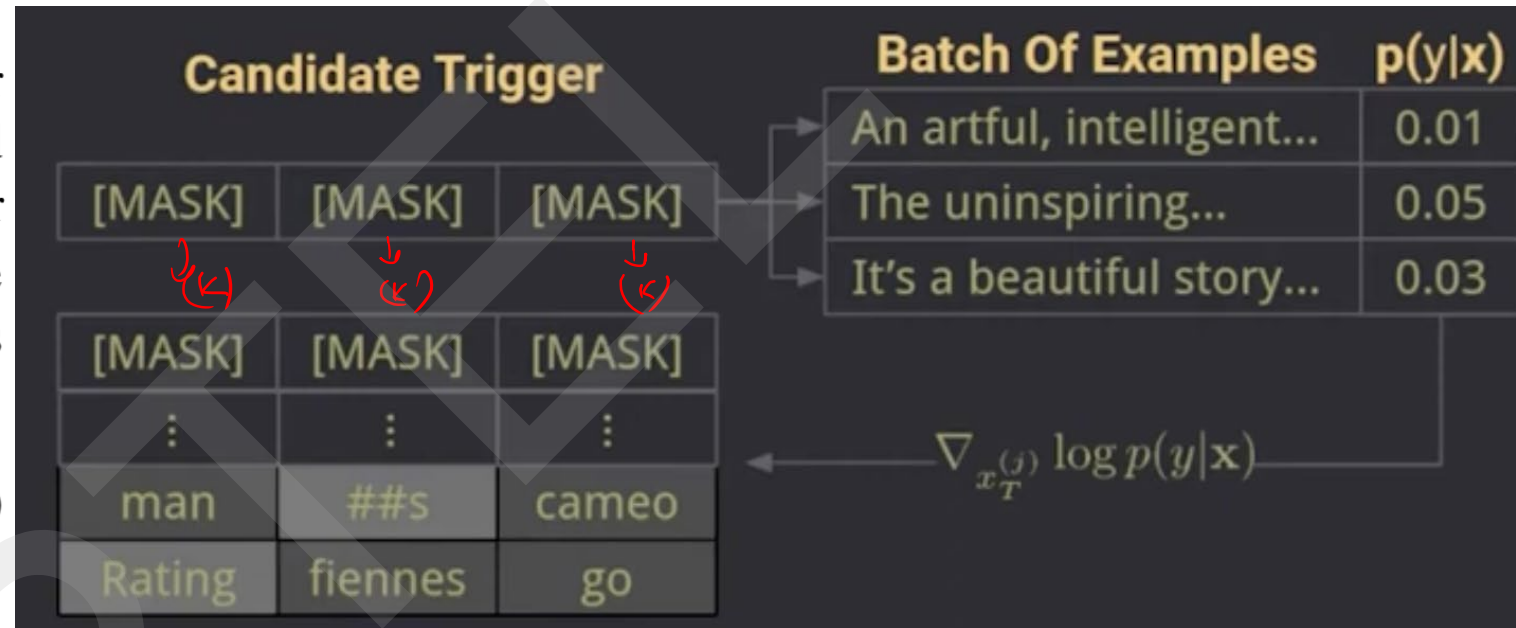
$$f(x + \varepsilon v) - f(x) \approx \varepsilon \nabla_v f(x).$$

In words, we can interpret this as follows: If we start at  $x$  and move to  $x + \varepsilon v$ , then the value of  $f$  increases by approximately  $\varepsilon \nabla_v f(x)$ .

# Gradient-based discrete prompt search

Formally, at each step, we compute a first-order approximation of the change in the log-likelihood that would be produced by swapping the  $j$ th trigger token  $x_{\text{trig}}^{(j)}$  with another token  $w \in \mathcal{V}$ . Then we identify a candidate set  $\mathcal{V}_{\text{cand}}$  of the top- $k$  tokens estimated to cause the greatest increase:

$$\mathcal{V}_{\text{cand}} = \underset{w \in \mathcal{V}}{\text{top-}k} [\mathbf{w}_{\text{in}}^T \nabla \log p(y|\mathbf{x}_{\text{prompt}})] \quad (2)$$

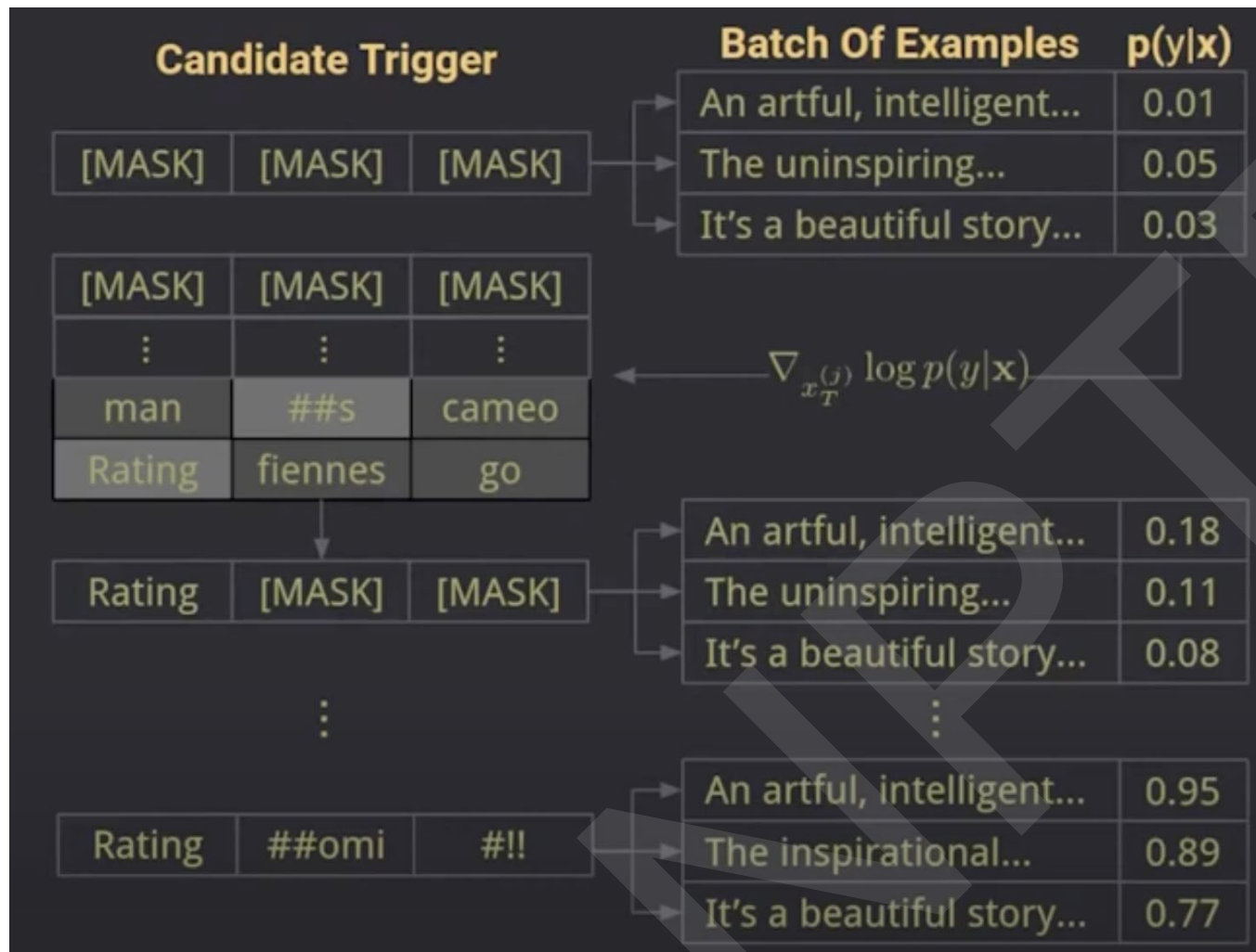


For each candidate in this set, we then re-evaluate the equation on the updated prompt, and retain the prompt with the highest probability in the next step

$$p(y|\mathbf{x}_{\text{prompt}}) = \sum_{w \in \mathcal{V}_y} p([\text{MASK}] = w | \mathbf{x}_{\text{prompt}})$$

$\mathcal{V}_y$  is a set of label tokens that correspond to a particular label  $y$

# Gradient-based discrete prompt search: beam search



We augment this token replacement strategy with beam search. We consider the top-k token candidates for each token position in the trigger. We search left to right across the positions and score each beam using

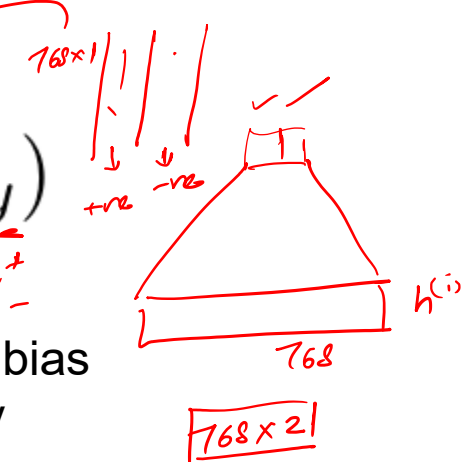
$$p(y|\mathbf{x}_{\text{prompt}}) = \sum_{w \in \mathcal{V}_y} p([\text{MASK}] = w | \mathbf{x}_{\text{prompt}})$$

# Automating label token selection

We develop a general two-step approach to automate the selection of the sets of label tokens  $V_y$ .

In the **first step**, we train a logistic classifier to predict the class label using the contextualized embedding of the [MASK] token as input:  $h = \text{Transformer}_{\text{enc}}(\tilde{x})$

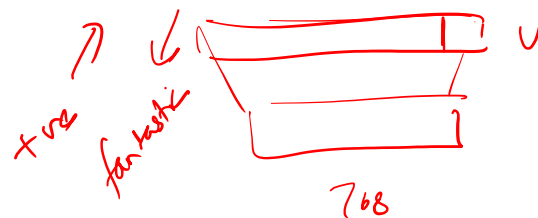
write the output of this classifier as:  $p(y|h^{(i)}) \propto \exp(\underbrace{h^{(i)}}_{\substack{\downarrow \\ c^+ \\ c^-}} \cdot \underbrace{y}_{\substack{\uparrow \\ c^+ \\ c^-}} + \underbrace{\beta_y}_{\substack{\text{learned weight and bias terms for the label } y}})$



In the **second step**, we substitute  $h^{(i)}$  with the MLM's output word embeddings  $w_{\text{out}}$  to obtain

$$s(y, w) = p(y|w_{\text{out}})$$

$$\mathcal{V}_y = \underset{w \in \mathcal{V}}{\text{top-}k} [s(y, w)]$$



I/p ~~Prompt~~ [MASK]

$$\boxed{w_{\text{out}} \cdot c^+} + \beta_{c^+}$$

# Try this problem

Suppose that you are using autoprompt with BERT for text classification with 2 classes, positive and negative. Explain the two steps required to decide on the set of label words for each class. What are the number of learnable parameters? Assume that you want to use  $V_y = 3$

$$p(y|\mathbf{h}^{(i)}) \propto \exp(\mathbf{h}^{(i)} \cdot \mathbf{y} + \beta_y)$$

$$s(y, w) = p(y|\mathbf{w}_{\text{out}})$$

$$\mathcal{V}_y = \underset{w \in \mathcal{V}}{\text{top-}k} [s(y, w)]$$

768  $\begin{cases} +ve \\ -ve \end{cases}$   
 $768 \times 2 + 2$

# Huge gains in performance

| Model                    | Dev  | Test              |
|--------------------------|------|-------------------|
| BiLSTM                   | -    | 82.8 <sup>†</sup> |
| BiLSTM + ELMo            | -    | 89.3 <sup>†</sup> |
| BERT (linear probing)    | 85.2 | 83.4              |
| BERT (finetuned)         | -    | 93.5 <sup>†</sup> |
| RoBERTa (linear probing) | 87.9 | 88.8              |
| RoBERTa (finetuned)      | -    | 96.7 <sup>†</sup> |
| BERT (manual)            | 63.2 | 63.2              |
| BERT (AUTOPROMPT)        | 80.9 | 82.3              |
| RoBERTa (manual)         | 85.3 | 85.2              |
| RoBERTa (AUTOPROMPT)     | 91.2 | 91.4              |

# Example Prompts

| Task                | Prompt Template                                           | Prompt found by AUTOPROMPT                                                                                                                          | Label Tokens                                                                                |
|---------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Sentiment Analysis  | {sentence} [T]...[T] [P].                                 | unflinchingly bleak and desperate<br>Writing academicswhere overseas<br>will appear [MASK].                                                         | pos: partnership, extraordinary, ##bla<br>neg: worse, persisted, unconstitutional           |
| NLI                 | {prem}[P][T]...[T]{hyp}<br><i>/ P ~~~~~</i>               | Two dogs are wrestling and<br>hugging [MASK] concretepathic<br>workplace There is no dog<br>wrestling and hugging                                   | con: Nobody, nobody, nor<br>ent: ##found, ##ways, Agency<br>neu: ##ponents, ##lary, ##uated |
| Fact Retrieval      | <i>X plays Y music</i><br>{sub}[T]...[T][P].              | Hall Overton fireplacemade antique<br>son alto [MASK].                                                                                              |                                                                                             |
| Relation Extraction | <i>X is a Y by profession</i><br>{sent}{sub}[T]...[T][P]. | Leonard Wood (born February 4,<br>1942) is a former Canadian<br>politician.<br>Leonard Wood gymnasium<br>brotherdicative himself another<br>[MASK]. |                                                                                             |

# EvoPrompt

## Genetic Algorithm (GA) Implemented by LLMs

### Query:

Please follow the instruction step-by-step to generate a better prompt.

1. **Cross over** the following prompts and generate a new prompt:

Prompt 1: Now you are a categorizer, your mission is to ascertain the sentiment of the provided text, either favorable or unfavourable.

Prompt 2: Assign a sentiment label to the given sentence from ['negative', 'positive'] and return only the label without any other text.

2. **Mutate** the prompt generated in Step 1 and generate a final prompt bracketed with <prompt> and </prompt>.

### Response:

1. **Crossover** Prompt: Your mission is to ascertain the sentiment of the provided text and assign a sentiment label from ['negative', 'positive'].

2. <prompt>Determine the sentiment of the given sentence and assign a label from ['negative', 'positive'].</prompt>

Crossover

Mutate

In Step 1, LLMs perform crossover on the given two prompts (words in orange and blue are inherited from Prompt 1 and Prompt 2, respectively). In Step 2, LLMs perform mutation on the prompt.

# EvoPrompt

## Differential Evolution (DE) Algorithm Implemented by LLMs

### Query:

Please follow the instruction step-by-step to generate a better prompt.

1. Identify the **different parts** between the Prompt 1 and Prompt 2:

Prompt 1: Categorize the tweet according to if it has a positive or negative sentiment.

Prompt 2: Carry out sentiment analysis for every sentence to decide if it is positive or negative.

2. Randomly **mutate** the different parts

3. Combine the different parts with Prompt 3, selectively replace it with the different parts in Step 2 and generate a new prompt.

Prompt 3: In this task, you are given sentences from product reviews. The task is to classify a sentence as positive or as negative.

4. **Cross over** the prompt in the Step 3 with the following basic prompt and generate a final prompt bracketed with <prompt> and </prompt>:

Basic Prompt: Here, you'll be given sentences from reviews about products and you'll need to decide if it's a positive or a negative review.

### Response:

1. Different parts:

"tweet" vs "sentence"  
"Categorize" vs "Carry out sentiment analysis"

2. "tweet" -> "review"

"Categorize" -> "Analyze"

"Sentiment analysis" -> "Sentiment identification"

$F(b - c)$

3. New Prompt: In this task, you are given reviews about products. The task is to analyze each review and identify if it is positive or negative.

$a + F(b - c)$

4. Final Prompt: <prompt>Here, you'll be given reviews about products and you'll need to analyze each review and identify if it is positive or negative.</prompt>

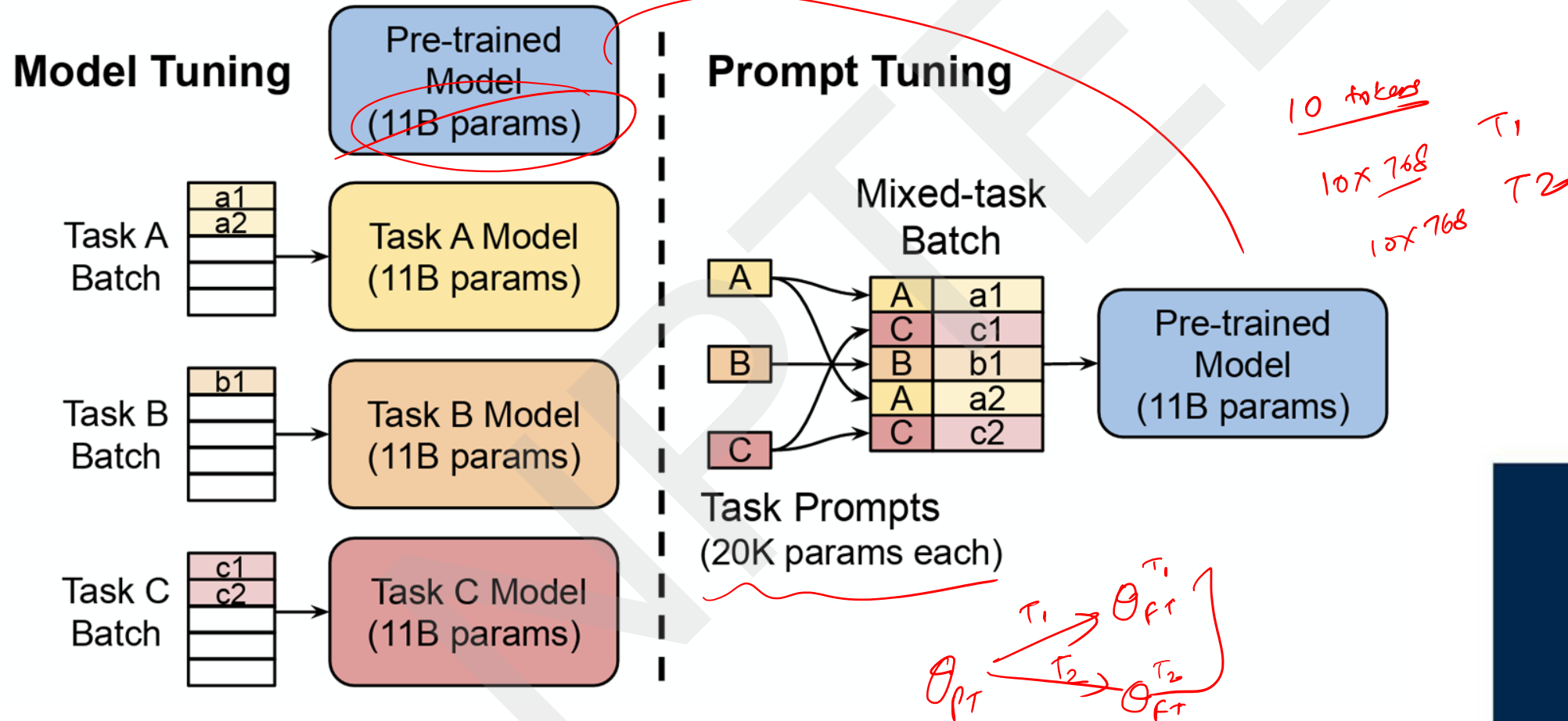
Crossover

We evaluate the generated candidate prompts on a development set and retain those with superior performance, similar to the survival of the fittest in nature.

Connecting Large Language Models with Evolutionary Algorithms Yields Powerful Prompt Optimizers: <https://arxiv.org/pdf/2309.08532>

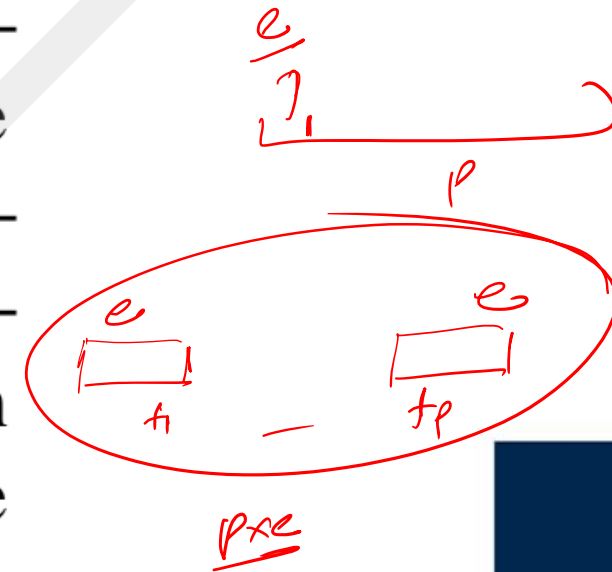
# Prompt-tuning

Optimize the embeddings of a prompt, instead of the words.

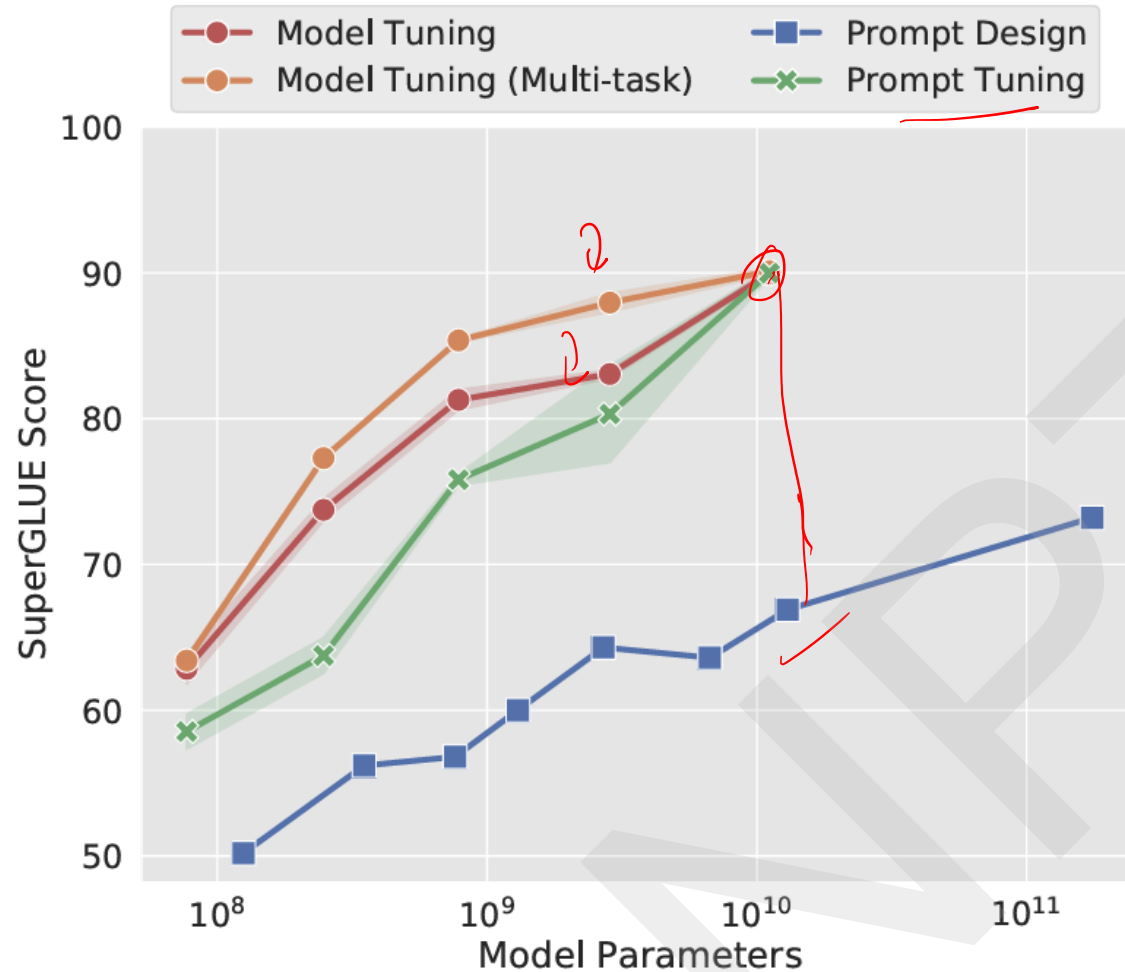


# Prompt-tuning: So, how does it work?

Given a series of  $n$  tokens,  $\{x_1, x_2, \dots, x_n\}$ , the first thing T5 does is embed the tokens, forming a matrix  $X_e \in \mathbb{R}^{n \times e}$  where  $e$  is the dimension of the embedding space. Our soft-prompts are represented as a parameter  $P_e \in \mathbb{R}^{p \times e}$ , where  $p$  is the length of the prompt. Our prompt is then concatenated to the embedded input forming a single matrix  $[P_e; X_e] \in \mathbb{R}^{(p+n) \times e}$  which then flows through the encoder-decoder as normal. Our models are trained to maximize the probability of  $Y$ , but only the prompt parameters  $P_e$  are updated.



# Prompt-tuning: How effective?



Standard **model tuning** of T5 achieves strong performance, but requires storing separate copies of the model for each end task. Our prompt tuning of T5 matches the quality of model tuning as size increases, while enabling the reuse of a single frozen model for all tasks. Our approach significantly outperforms fewshot prompt design using GPT-3. We show mean and standard deviation across 3 runs for tuning methods.

*Will talk about more parameter-efficient methods next week*

# Try this problem

Suppose you want to use T5 model, which has a model dimensionality of 1024 and 24 layers, for the tasks of: (1). text summarization, (2). question answering and (3). text simplification. You can either use full fine-tuning for each task or prompt-tuning (soft-prompts of length 10 for each task). How many extra parameters will you have to store in both of these scenarios at inference time, apart from the T5 parameters?

Handwritten calculations for parameter counts:

- Encoder:  $12d^2 \times 24$
- Decoder:  $16d^2 \times 2$
- Full FT:  $\rightarrow$  full FT
- T5:  $28 \times (1024)^2 \times 24$
- Prompt-tuning:  $2 \times (1024 \times 10) \times 3$

# REFERENCES

Kamath, Uday, et al. "Large Language Models: A Deep Dive." (2024).



**THANK YOU**